

基于深度视觉模型的智能文档扫描识别软件源代码

软件名称：基于深度视觉模型的智能文档扫描识别软件 V1.0

版本号：V1.0

文档类型：源代码

目录

1. mobile/android/app/src/debug/AndroidManifest.xml
2. mobile/android/app/src/main/AndroidManifest.xml
3. mobile/android/app/src/main/java/io/flutter/plugins/GeneratedPluginRegistrant.java
4. mobile/android/app/src/main/kotlin/com/dsocr/ds_ocr_mobile/MainActivity.kt
5. mobile/android/app/src/main/res/drawable-v21/launch_background.xml
6. mobile/android/app/src/main/res/drawable/launch_background.xml
7. mobile/android/app/src/main/res/values-night/styles.xml
8. mobile/android/app/src/main/res/values/styles.xml
9. mobile/android/app/src/profile/AndroidManifest.xml
10. mobile/lib/main.dart
11. mobile/lib/src/api/ocr_api.dart
12. mobile/lib/src/app.dart
13. mobile/lib/src/features/ocr_home_page.dart
14. mobile/lib/src/models/ocr_models.dart
15. mobile/lib/src/settings/settings_store.dart
16. scripts/prefetch_models.py
17. scripts/run_deepseek_ocr2.py
18. src/ds_ocr_runner/__init__.py
19. src/ds_ocr_runner/api.py
20. src/ds_ocr_runner/config.py
21. src/ds_ocr_runner/db.py
22. src/ds_ocr_runner/main.py
23. src/ds_ocr_runner/manage.py
24. src/ds_ocr_runner/models.py
25. src/ds_ocr_runner/ocr_backend.py
26. src/ds_ocr_runner/schemas.py
27. src/ds_ocr_runner/services.py
28. src/ds_ocr_runner/storage.py
29. src/ds_ocr_runner/worker.py
30. src/ds_ocr2_3090_runner.egg-info/dependency_links.txt
31. src/ds_ocr2_3090_runner.egg-info/requires.txt
32. src/ds_ocr2_3090_runner.egg-info/SOURCES.txt
33. src/ds_ocr2_3090_runner.egg-info/top_level.txt
34. compose.yaml
35. Dockerfile
36. mobile/pubspec.yaml

mobile/android/app/src/debug/AndroidManifest.xml

```
<manifest xmlns:android="***"
  <!-- The INTERNET permission is required for development. Specifically,
    the Flutter tool needs it to communicate with the running application
    to allow setting breakpoints, to provide hot reload, etc.
  -->
  <uses-permission android:name="android.permission.INTERNET"/>
</manifest>
```

mobile/android/app/src/main/AndroidManifest.xml

```
<manifest xmlns:android="***"
  <uses-permission android:name="android.permission.INTERNET"/>

  <application
    android:label="DS OCR"
    android:name="{applicationName}"
    android:icon="@mipmap/ic_launcher"
    android:usesCleartextTraffic="true">
    <activity
      android:name=".MainActivity"
      android:exported="true"
      android:launchMode="singleTop"
      android:taskAffinity=""
      android:theme="@style/LaunchTheme"

android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smallestScreenSize|locale|layoutDirection|fontScale|screenLayout|density|uiMode"
      android:hardwareAccelerated="true"
      android:windowSoftInputMode="adjustResize">
      <!-- Specifies an Android theme to apply to this Activity as soon as
        the Android process has started. This theme is visible to the user
        while the Flutter UI initializes. After that, this theme continues
        to determine the Window background behind the Flutter UI. -->
      <meta-data
        android:name="io.flutter.embedding.android.NormalTheme"
        android:resource="@style/NormalTheme"
      />
      <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
    <!-- Don't delete the meta-data below.
      This is used by the Flutter tool to generate GeneratedPluginRegistrant.java -->
    <meta-data
      android:name="flutterEmbedding"
      android:value="2" />
  </application>
  <!-- Required to query activities that can process text, see:
    *** and
    ***

    In particular, this is used by the Flutter engine in io.flutter.plugin.text.ProcessTextPlugin. -->
  <queries>
    <intent>
      <action android:name="android.intent.action.PROCESS_TEXT"/>
      <data android:mimeType="text/plain"/>
    </intent>
  </queries>
</manifest>
```

mobile/android/app/src/main/java/io/flutter/plugins/GeneratedPluginRegistrant.java

```
package io.flutter.plugins;

import androidx.annotation.Keep;
import androidx.annotation.NonNull;
import io.flutter.Log;

import io.flutter.embedding.engine.FlutterEngine;

/**
 * Generated file. Do not edit.
 * This file is generated by the Flutter tool based on the
 * plugins that support the Android platform.
 */
@Keep
public final class GeneratedPluginRegistrant {
    private static final String TAG = "GeneratedPluginRegistrant";
    public static void registerWith(@NonNull FlutterEngine flutterEngine) {
        try {
            flutterEngine.getPlugins().add(new com.mr.flutter.plugin.filepicker.FilePickerPlugin());
        } catch (Exception e) {
            Log.e(TAG, "Error registering plugin file_picker, com.mr.flutter.plugin.filepicker.FilePickerPlugin", e);
        }
        try {
            flutterEngine.getPlugins().add(new
io.flutter.plugins.flutter_plugin_android_lifecycle.FlutterAndroidLifecyclePlugin());
        } catch (Exception e) {
            Log.e(TAG, "Error registering plugin flutter_plugin_android_lifecycle,
io.flutter.plugins.flutter_plugin_android_lifecycle.FlutterAndroidLifecyclePlugin", e);
        }
        try {
            flutterEngine.getPlugins().add(new io.flutter.plugins.sharedpreferences.SharedPreferencesPlugin());
        } catch (Exception e) {
            Log.e(TAG, "Error registering plugin shared_preferences_android,
io.flutter.plugins.sharedpreferences.SharedPreferencesPlugin", e);
        }
    }
}
```

mobile/android/app/src/main/kotlin/com/dsocr/ds_ocr_mobile/MainActivity.kt

```
package com.dsocr.ds_ocr_mobile

import io.flutter.embedding.android.FlutterActivity

class MainActivity : FlutterActivity()
```

mobile/android/app/src/main/res/drawable-v21/launch_background.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Modify this file to customize your launch splash screen -->
<layer-list xmlns:android="***"
    <item android:drawable="?android:colorBackground" />

    <!-- You can insert your own image assets here -->
    <!-- <item>
        <bitmap
            android:gravity="center"
            android:src="@mipmap/launch_image" />
    </item> -->
</layer-list>
```

mobile/android/app/src/main/res/drawable/launch_background.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Modify this file to customize your launch splash screen -->
<layer-list xmlns:android="***"
    <item android:drawable="@android:color/white" />

    <!-- You can insert your own image assets here -->
    <!-- <item>
        <bitmap
            android:gravity="center"
            android:src="@mipmap/launch_image" />
    </item> -->
</layer-list>
```

mobile/android/app/src/main/res/values-night/styles.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Theme applied to the Android Window while the process is starting when the OS's Dark Mode setting is on -->
    <style name="LaunchTheme" parent="@android:style/Theme.Black.NoTitleBar">
        <!-- Show a splash screen on the activity. Automatically removed when
             the Flutter engine draws its first frame -->
        <item name="android:windowBackground">@drawable/launch_background</item>
    </style>
    <!-- Theme applied to the Android Window as soon as the process has started.
         This theme determines the color of the Android Window while your
         Flutter UI initializes, as well as behind your Flutter UI while its
         running.

         This Theme is only used starting with V2 of Flutter's Android embedding. -->
    <style name="NormalTheme" parent="@android:style/Theme.Black.NoTitleBar">
        <item name="android:windowBackground">?android:colorBackground</item>
    </style>
</resources>
```

mobile/android/app/src/main/res/values/styles.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Theme applied to the Android Window while the process is starting when the OS's Dark Mode setting is off -
->
    <style name="LaunchTheme" parent="@android:style/Theme.Light.NoTitleBar">
        <!-- Show a splash screen on the activity. Automatically removed when
             the Flutter engine draws its first frame -->
        <item name="android:windowBackground">@drawable/launch_background</item>
    </style>
    <!-- Theme applied to the Android Window as soon as the process has started.
         This theme determines the color of the Android Window while your
         Flutter UI initializes, as well as behind your Flutter UI while its
         running.

         This Theme is only used starting with V2 of Flutter's Android embedding. -->
    <style name="NormalTheme" parent="@android:style/Theme.Light.NoTitleBar">
        <item name="android:windowBackground">?android:colorBackground</item>
    </style>
</resources>
```

mobile/android/app/src/profile/AndroidManifest.xml

```
<manifest xmlns:android="***"
    <!-- The INTERNET permission is required for development. Specifically,
         the Flutter tool needs it to communicate with the running application
         to allow setting breakpoints, to provide hot reload, etc.
    -->
```

```
<uses-permission android:name="android.permission.INTERNET"/>
</manifest>
```

mobile/lib/main.dart

```
import 'package:flutter/material.dart';

import 'src/app.dart';

void main() {
  runApp(const DsOcrApp());
}
```

mobile/lib/src/api/ocr_api.dart

```
import 'dart:convert';
import 'dart:io';

import 'package:http/http.dart' as http;
import '../models/ocr_models.dart';

class OcrApi {
  OcrApi({required this.baseUrl, http.Client? client})
    : _client = client ?? http.Client();

  final String baseUrl;
  final http.Client _client;

  Uri _uri(String path) =>
    Uri.parse('${baseUrl.replaceAll(RegExp(r'/' + '$'), '')} $path');

  Future<OcrFile> uploadFile(File file) async {
    final request = http.MultipartRequest('POST', _uri('/api/files'));
    request.files.add(await http.MultipartFile.fromPath('upload', file.path));

    final streamed = await _client.send(request);
    final response = await http.Response.fromStream(streamed);
    _ensureSuccess(response);
    return OcrFile.fromJson(_decode(response));
  }

  Future<OcrTask> createTask({
    required String fileId,
    String mode = 'document_markdown',
    int priority = 100,
  }) async {
    final response = await _client.post(
      _uri('/api/ocr/tasks'),
      headers: {'content-type': 'application/json'},
      body: jsonEncode({
        'file_id': fileId,
        'mode': mode,
        'priority': priority,
        'options': {'preserve_layout': true, 'table': true, 'formula': true},
      })),
    );
    _ensureSuccess(response);
    return OcrTask.fromJson(_decode(response));
  }

  Future<OcrTask> getTask(String taskId) async {
    final response = await _client.get(_uri('/api/ocr/tasks/$taskId'));
    _ensureSuccess(response);
    return OcrTask.fromJson(_decode(response));
  }

  Future<OcrResult> getResult(String taskId) async {
```

```

    final response = await _client.get(_uri('/api/ocr/tasks/$taskId/result'));
    _ensureSuccess(response);
    return OcrResult.fromJson(_decode(response));
  }

  Future<OcrTask> cancelTask(String taskId) async {
    final response = await _client.post(_uri('/api/ocr/tasks/$taskId/cancel'));
    _ensureSuccess(response);
    return OcrTask.fromJson(_decode(response));
  }

  Map<String, dynamic> _decode(http.Response response) {
    return jsonDecode(utf8.decode(response.bodyBytes)) as Map<String, dynamic>;
  }

  void _ensureSuccess(http.Response response) {
    if (response.statusCode >= 200 && response.statusCode < 300) {
      return;
    }

    String message = response.body;
    try {
      final decoded = _decode(response);
      message = decoded['detail']?.toString() ?? message;
    } catch (_) {
      // Keep the raw body when the backend does not return JSON.
    }
    throw OcrApiException(response.statusCode, message);
  }
}

class OcrApiException implements Exception {
  OcrApiException(this.statusCode, this.message);

  final int statusCode;
  final String message;

  @override
  String toString() => 'HTTP $statusCode: $message';
}

```

mobile/lib/src/app.dart

```

import 'package:flutter/material.dart';
import 'features/ocr_home_page.dart';

class DsOcrApp extends StatelessWidget {
  const DsOcrApp({super.key});

  @override
  Widget build(BuildContext context) {
    const seed = Color(0xFF256D85);

    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'DS OCR',
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: seed,
          brightness: Brightness.light,
        ),
      ),
      scaffoldBackgroundColor: const Color(0xFFF7F8FA),
      appBarTheme: const AppBarTheme(
        centerTitle: false,
        backgroundColor: Color(0xFFF7F8FA),
        surfaceTintColor: Colors.transparent,
      ),
    ),
  }

```

```

    ),
    home: const OcrHomePage(),
  );
}
}

```

mobile/lib/src/features/ocr_home_page.dart

```

import 'dart:io';

import 'package:file_picker/file_picker.dart';
import 'package:flutter/material.dart';

import '../api/ocr_api.dart';
import '../models/ocr_models.dart';
import '../settings/settings_store.dart';

class OcrHomePage extends StatefulWidget {
  const OcrHomePage({super.key});

  @override
  State<OcrHomePage> createState() => _OcrHomePageState();
}

class _OcrHomePageState extends State<OcrHomePage> {
  final _settings = SettingsStore();
  final _baseUrlController = TextEditingController();
  final List<OcrJob> _jobs = [];

  PlatformFile? _selectedFile;
  OcrTask? _currentTask;
  OcrResult? _result;
  bool _busy = false;
  String _baseUrl = SettingsStore.defaultBaseUrl;
  String _stage = 'Idle';
  String? _error;
  int _resultTab = 0;

  OcrApi get _api => OcrApi(baseUrl: _baseUrl);

  @override
  void initState() {
    super.initState();
    _loadSettings();
  }

  @override
  void dispose() {
    _baseUrlController.dispose();
    super.dispose();
  }

  Future<void> _loadSettings() async {
    final baseUrl = await _settings.loadBaseUrl();
    if (!mounted) {
      return;
    }
    setState(() {
      _baseUrl = baseUrl;
      _baseUrlController.text = baseUrl;
    });
  }

  Future<void> _pickFile() async {
    final result = await FilePicker.platform.pickFiles(
      type: FileType.custom,
      allowedExtensions: const ['jpg', 'jpeg', 'png', 'webp', 'pdf'],
      withData: false,
    );
  }

```

```

final file = result?.files.single;
if (file == null) {
  return;
}

setState(() {
  _selectedFile = file;
  _currentTask = null;
  _result = null;
  _error = null;
  _stage = 'Ready';
});
}

Future<void> _submit() async {
  final path = _selectedFile?.path;
  if (path == null) {
    setState(() => _error = 'Select a file first.');
```

return;

```

  }

  setState(() {
    _busy = true;
    _error = null;
    _result = null;
    _stage = 'Uploading';
  });

  try {
    final api = _api;
    final file = await api.uploadFile(File(path));
    setState(() => _stage = 'Creating task');
```

var task = await api.createTask(fileId: file.id);

```

    _rememberJob(file, task);
    setState(() {
      _currentTask = task;
      _stage = task.status;
    });

    task = await _pollTask(api, task.id);
    if (!mounted) {
      return;
    }
    setState(() {
      _currentTask = task;
      _stage = task.status;
    });
    _updateJob(task);

    if (task.status == 'succeeded') {
      final result = await api.getResult(task.id);
      if (!mounted) {
        return;
      }
      setState(() {
        _result = result;
        _stage = 'Completed';
      });
    } else if (task.errorMessage != null) {
      setState(() => _error = task.errorMessage);
    }
  } catch (error) {
    if (!mounted) {
      return;
    }
    setState(() {
      _error = error.toString();
      _stage = 'Failed';
    });
  }
}

```

```

    } finally {
      if (mounted) {
        setState(() => _busy = false);
      }
    }
  }
}

Future<OcrTask> _pollTask(OcrApi api, String taskId) async {
  var task = await api.getTask(taskId);
  for (var i = 0; i < 120 && !task.isTerminal; i++) {
    if (!mounted) {
      return task;
    }
    setState(() {
      _currentTask = task;
      _stage = task.status;
    });
    await Future<void>.delayed(const Duration(seconds: 2));
    task = await api.getTask(taskId);
  }
  return task;
}

void _rememberJob(OcrFile file, OcrTask task) {
  final job = OcrJob(
    fileName: file.originalName,
    fileId: file.id,
    taskId: task.id,
    status: task.status,
    createdAt: DateTime.now(),
  );
  setState(() => _jobs.insert(0, job));
}

void _updateJob(OcrTask task) {
  final index = _jobs.indexWhere((job) => job.taskId == task.id);
  if (index == -1) {
    return;
  }
  setState(() => _jobs[index] = _jobs[index].copyWith(status: task.status));
}

Future<void> _saveBaseUrl() async {
  final value = _baseUrlController.text.trim();
  if (value.isEmpty) {
    return;
  }
  await _settings.saveBaseUrl(value);
  if (!mounted) {
    return;
  }
  setState(() => _baseUrl = value);
  Navigator.of(context).pop();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('DS OCR'),
      actions: [
        IconButton(
          tooltip: 'Settings',
          onPressed: _openSettings,
          icon: const Icon(Icons.tune),
        ),
      ],
    ),
    body: SafeArea(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(16, 8, 16, 24),

```

```

        children: [
          _TaskPanel(
            selectedFile: _selectedFile,
            busy: _busy,
            stage: _stage,
            task: _currentTask,
            onPickFile: _pickFile,
            onSubmit: _submit,
          ),
          if (_error != null) ...[
            const SizedBox(height: 12),
            _ErrorBanner(message: _error!),
          ],
          const SizedBox(height: 12),
          _ResultPanel(
            result: _result,
            selectedIndex: _resultTab,
            onTabChanged: (index) => setState(() => _resultTab = index),
          ),
          const SizedBox(height: 12),
          _HistoryPanel(jobs: _jobs),
        ],
      ),
    ),
  );
}

void _openSettings() {
  _baseUrlController.text = _baseUrl;
  showModalBottomSheet<void>({
    context: context,
    isScrollControlled: true,
    showDragHandle: true,
    builder: (context) {
      return Padding(
        padding: EdgeInsets.fromLTRB(
          16,
          8,
          16,
          16 + MediaQuery.viewInsetsOf(context).bottom,
        ),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Text('Backend', style: Theme.of(context).textTheme.titleMedium),
            const SizedBox(height: 12),
            TextField(
              controller: _baseUrlController,
              keyboardType: TextInputType.url,
              decoration: const InputDecoration(
                labelText: 'API base URL',
                prefixIcon: Icon(Icons.link),
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 12),
            FilledButton.icon(
              onPressed: _saveBaseUrl,
              icon: const Icon(Icons.save),
              label: const Text('Save'),
            ),
          ],
        ),
      );
    },
  );
}
}

```

```

class _TaskPanel extends StatelessWidget {

```

```

const _TaskPanel({
  required this.selectedFile,
  required this.busy,
  required this.stage,
  required this.task,
  required this.onPickFile,
  required this.onSubmit,
});

final PlatformFile? selectedFile;
final bool busy;
final String stage;
final OcrTask? task;
final VoidCallback onPickFile;
final VoidCallback onSubmit;

@override
Widget build(BuildContext context) {
  final file = selectedFile;

  return Card(
    elevation: 0,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Row(
            children: [
              const Icon(Icons.document_scanner_outlined),
              const SizedBox(width: 8),
              Expanded(
                child: Text(
                  'OCR Task',
                  style: Theme.of(context).textTheme.titleMedium,
                ),
              ),
              _StatusChip(label: stage),
            ],
          ),
          const SizedBox(height: 16),
          Container(
            constraints: const BoxConstraints(minHeight: 72),
            padding: const EdgeInsets.all(12),
            decoration: BoxDecoration(
              border: Border.all(color: Theme.of(context).dividerColor),
              borderRadius: BorderRadius.circular(8),
            ),
            child: Row(
              children: [
                const Icon(Icons.insert_drive_file_outlined),
                const SizedBox(width: 12),
                Expanded(
                  child: Text(
                    file?.name ?? 'No file selected',
                    maxLines: 2,
                    overflow: TextOverflow.ellipsis,
                  ),
                ),
              ],
            ),
          ),
          const SizedBox(height: 12),
          if (task != null)
            Text(
              'Task ${task!.id} - attempts ${task!.attempts}/${task!.maxAttempts}',
              maxLines: 2,
              overflow: TextOverflow.ellipsis,
              style: Theme.of(context).textTheme.bodySmall,
            ),
        ],
      ),
    ),
  );
}

```

```

        if (task != null) const SizedBox(height: 12),
        Row(
          children: [
            Expanded(
              child: OutlinedButton.icon(
                onPressed: busy ? null : onPickFile,
                icon: const Icon(Icons.upload_file),
                label: const Text('Select'),
              ),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: FilledButton.icon(
                onPressed: busy ? null : onSubmit,
                icon: busy
                  ? const SizedBox.square(
                      dimension: 18,
                      child: CircularProgressIndicator(strokeWidth: 2),
                    )
                  : const Icon(Icons.play_arrow),
                label: const Text('Run OCR'),
              ),
            ),
          ],
        ),
      ],
    ),
  ),
);
}
}

```

```

class _ResultPanel extends StatelessWidget {
  const _ResultPanel({
    required this.result,
    required this.selectedIndex,
    required this.onTabChanged,
  });

  final OcrResult? result;
  final int selectedIndex;
  final ValueChanged<int> onTabChanged;

  @override
  Widget build(BuildContext context) {
    final content = result == null
      ? ''
      : selectedIndex == 0
      ? result!.markdown
      : result!.text;

    return Card(
      elevation: 0,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Row(
              children: [
                const Icon(Icons.article_outlined),
                const SizedBox(width: 8),
                Expanded(
                  child: Text(
                    'Result',
                    style: Theme.of(context).textTheme.titleMedium,
                  ),
                ),
              ],
            ),
            SegmentedButton<int>(
              segments: const [

```

```

        ButtonSegment(
          value: 0,
          icon: Icon(Icons.notes),
          label: Text('MD'),
        ),
        ButtonSegment(
          value: 1,
          icon: Icon(Icons.text_fields),
          label: Text('TXT'),
        ),
      ],
      selected: {selectedIndex},
      onSelectionChanged: (value) => onTabChanged(value.first),
      showSelectedIcon: false,
    ),
  ],
),
const SizedBox(height: 12),
Container(
  constraints: const BoxConstraints(minHeight: 220),
  padding: const EdgeInsets.all(12),
  decoration: BoxDecoration(
    color: Colors.white,
    border: Border.all(color: Theme.of(context).dividerColor),
    borderRadius: BorderRadius.circular(8),
  ),
  child: result == null
    ? const Center(
        child: Icon(Icons.description_outlined, size: 40),
      )
    : SingleChildScrollView(
        child: SelectableText(
          content.isEmpty ? '(empty)' : content,
          style: const TextStyle(
            fontFamily: 'monospace',
            fontSize: 13,
          ),
        ),
      ),
    ),
  ),
),
],
),
);
}
}

```

```

class _HistoryPanel extends StatelessWidget {
  const _HistoryPanel({required this.jobs});

  final List<OcrJob> jobs;

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 0,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            Row(
              children: [
                const Icon(Icons.history),
                const SizedBox(width: 8),
                Text('Recent', style: Theme.of(context).textTheme.titleMedium),
              ],
            ),
            const SizedBox(height: 8),
            if (jobs.isEmpty)

```

```

        const SizedBox(height: 72, child: Center(child: Text('No tasks')))
      else
        for (final job in jobs.take(8))
          ListTile(
            contentPadding: EdgeInsets.zero,
            title: Text(
              job.fileName,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
            ),
            subtitle: Text(
              job.taskId,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
            ),
            trailing: _StatusChip(label: job.status),
          ),
    ],
  ),
),
);
}
}

```

```

class _StatusChip extends StatelessWidget {
  const _StatusChip({required this.label});

  final String label;

  @override
  Widget build(BuildContext context) {
    final color = switch (label.toLowerCase()) {
      'succeeded' || 'completed' => Colors.green,
      'failed' => Colors.red,
      'running' || 'uploading' || 'creating task' => Colors.blue,
      'queued' => Colors.orange,
      _ => Colors.grey,
    };

    return Chip(
      visualDensity: VisualDensity.compact,
      side: BorderSide(color: color.withValues(alpha: 0.3)),
      backgroundColor: color.withValues(alpha: 0.08),
      label: Text(label, overflow: TextOverflow.ellipsis),
    );
  }
}

```

```

class _ErrorBanner extends StatelessWidget {
  const _ErrorBanner({required this.message});

  final String message;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: Colors.red.withValues(alpha: 0.08),
        border: Border.all(color: Colors.red.withValues(alpha: 0.25)),
        borderRadius: BorderRadius.circular(8),
      ),
      child: Row(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Icon(Icons.error_outline, color: Colors.red),
          const SizedBox(width: 8),
          Expanded(child: Text(message)),
        ],
      ),
    );
  }
}

```

```
}  
}
```

mobile/lib/src/models/ocr_models.dart

```
class OcrFile {  
  const OcrFile({  
    required this.id,  
    required this.originalName,  
    required this.contentType,  
    required this.sizeBytes,  
    required this.sha256,  
    required this.createdAt,  
  });  
  
  final String id;  
  final String originalName;  
  final String? contentType;  
  final int sizeBytes;  
  final String sha256;  
  final DateTime createdAt;  
  
  factory OcrFile.fromJson(Map<String, dynamic> json) {  
    return OcrFile(  
      id: json['id'] as String,  
      originalName: json['original_name'] as String,  
      contentType: json['content_type'] as String?,  
      sizeBytes: json['size_bytes'] as int,  
      sha256: json['sha256'] as String,  
      createdAt: DateTime.parse(json['created_at'] as String),  
    );  
  }  
}  
  
class OcrTask {  
  const OcrTask({  
    required this.id,  
    required this.fileId,  
    required this.mode,  
    required this.status,  
    required this.attempts,  
    required this.maxAttempts,  
    required this.errorMessage,  
    required this.createdAt,  
    required this.updatedAt,  
    required this.startedAt,  
    required this.completedAt,  
  });  
  
  final String id;  
  final String fileId;  
  final String mode;  
  final String status;  
  final int attempts;  
  final int maxAttempts;  
  final String? errorMessage;  
  final DateTime createdAt;  
  final DateTime updatedAt;  
  final DateTime? startedAt;  
  final DateTime? completedAt;  
  
  bool get isTerminal => {'succeeded', 'failed', 'canceled'}.contains(status);  
  
  factory OcrTask.fromJson(Map<String, dynamic> json) {  
    return OcrTask(  
      id: json['id'] as String,  
      fileId: json['file_id'] as String,  
      mode: json['mode'] as String,  
      status: json['status'] as String,
```

```

        attempts: json['attempts'] as int,
        maxAttempts: json['max_attempts'] as int,
        errorMessage: json['error_message'] as String?,
        createdAt: DateTime.parse(json['created_at'] as String),
        updatedAt: DateTime.parse(json['updated_at'] as String),
        startedAt: _parseDate(json['started_at']),
        completedAt: _parseDate(json['completed_at']),
    );
}

static DateTime? _parseDate(Object? value) {
    if (value == null) {
        return null;
    }
    return DateTime.parse(value as String);
}

}

class OcrResult {
    const OcrResult({
        required this.id,
        required this.taskId,
        required this.text,
        required this.markdown,
        required this.createdAt,
    });

    final String id;
    final String taskId;
    final String text;
    final String markdown;
    final DateTime createdAt;

    factory OcrResult.fromJson(Map<String, dynamic> json) {
        return OcrResult(
            id: json['id'] as String,
            taskId: json['task_id'] as String,
            text: json['text'] as String,
            markdown: json['markdown'] as String,
            createdAt: DateTime.parse(json['created_at'] as String),
        );
    }
}

class OcrJob {
    const OcrJob({
        required this.fileName,
        required this.fileId,
        required this.taskId,
        required this.status,
        required this.createdAt,
    });

    final String fileName;
    final String fileId;
    final String taskId;
    final String status;
    final DateTime createdAt;

    OcrJob copyWith({String? status}) {
        return OcrJob(
            fileName: fileName,
            fileId: fileId,
            taskId: taskId,
            status: status ?? this.status,
            createdAt: createdAt,
        );
    }
}

```

mobile/lib/src/settings/settings_store.dart

```
import 'package:shared_preferences/shared_preferences.dart';

class SettingsStore {
  static const _baseUrlKey = 'api_base_url';
  static const defaultBaseUrl = '***'

  Future<String> loadBaseUrl() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_baseUrlKey) ?? defaultBaseUrl;
  }

  Future<void> saveBaseUrl(String value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_baseUrlKey, value.trim());
  }
}
```

scripts/prefetch_models.py

```
import argparse
import os
from pathlib import Path

from huggingface_hub import HfApi, snapshot_download

DEFAULT_REPO_ID = "deepseek-ai/DeepSeek-OCR-2"
DEFAULT_REVISION = "aaa02f3811945a91062062994c5c4a3f4c0af2b0"
DEFAULT_ENDPOINT = "***"
DEFAULT_LOCAL_DIR = Path("models") / "DeepSeek-OCR-2"
PROXY_ENV_KEYS = ***
    "HTTP_PROXY",
    "HTTPS_PROXY",
    "ALL_PROXY",
    "http_proxy",
    "https_proxy",
    "all_proxy",
)

def disable_proxy_env() -> None:
    for key in PROXY_ENV_KEYS:
        os.environ.pop(key, None)

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Cache model snapshots without loading GPU code.")
    parser.add_argument("--repo-id", default=DEFAULT_REPO_ID)
    parser.add_argument("--revision", default=DEFAULT_REVISION)
    parser.add_argument("--endpoint", default=DEFAULT_ENDPOINT)
    parser.add_argument("--local-dir", default=str(DEFAULT_LOCAL_DIR))
    parser.add_argument("--max-workers", type=int, default=4)
    parser.add_argument("--keep-proxy", action="store_true")
    parser.add_argument("--dry-run", action="store_true")
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    if not args.keep_proxy:
        disable_proxy_env()

    local_dir = Path(args.local_dir).expanduser().resolve()
    local_dir.parent.mkdir(parents=True, exist_ok=True)

    api = HfApi(endpoint=args.endpoint)
    info = api.model_info(args.repo_id, revision=args.revision)
```

```

print(f"repo: {info.id}")
print(f"revision: {info.sha}")
print(f"files: {len(info.siblings)}")
print(f"target: {local_dir}")

if args.dry_run:
    return

path = snapshot_download(
    repo_id=args.repo_id,
    revision=args.revision,
    endpoint=args.endpoint,
    local_dir=local_dir,
    max_workers=args.max_workers,
)
print(f"cached: {path}")

if __name__ == "__main__":
    main()

```

scripts/run_deepseek_ocr2.py

```

import argparse
import json
import os
from pathlib import Path

import torch
from transformers import AutoModel, AutoTokenizer

DEFAULT_PROMPTS = {
    "document_markdown": "<image>\n<|grounding|>Convert the document to markdown. ",
    "plain_text": "<image>\nFree OCR. ",
    "free_ocr": "<image>\nFree OCR. ",
}
RESULT_PREFIX = "__DS_OCR_RESULT__="

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Run DeepSeek-OCR-2 inference on one image.")
    parser.add_argument("--model-dir", default="models/DeepSeek-OCR-2")
    parser.add_argument("--image", required=True)
    parser.add_argument("--output-dir", required=True)
    parser.add_argument("--mode", default="document_markdown")
    parser.add_argument("--prompt", default="")
    parser.add_argument("--base-size", type=int, default=1024)
    parser.add_argument("--image-size", type=int, default=768)
    parser.add_argument("--crop-mode", action=argparse.BooleanOptionalAction, default=True)
    parser.add_argument("--attn-implementation", default="eager")
    parser.add_argument("--dtype", choices=["bfloat16", "float16"], default="bfloat16")
    return parser.parse_args()

def dtype_from_name(name: str) -> torch.dtype:
    if name == "float16":
        return torch.float16
    return torch.bfloat16

def read_result(output_dir: Path, fallback: str | None) -> str:
    result_file = output_dir / "result.mmd"
    if result_file.is_file():
        return result_file.read_text(encoding="utf-8")
    return fallback or ""

def main() -> None:

```

```

args = parse_args()
model_dir = Path(args.model_dir).expanduser().resolve()
image_path = Path(args.image).expanduser().resolve()
output_dir = Path(args.output_dir).expanduser().resolve()
output_dir.mkdir(parents=True, exist_ok=True)

prompt = args.prompt or DEFAULT_PROMPTS.get(args.mode, DEFAULT_PROMPTS["document_markdown"])
dtype = dtype_from_name(args.dtype)

os.environ.setdefault("CUDA_VISIBLE_DEVICES", "0")
torch.backends.cuda.matmul.allow_tf32 = True

tokenizer = AutoTokenizer.from_pretrained(
    model_dir,
    trust_remote_code=True,
    local_files_only=True,
)
model = AutoModel.from_pretrained(
    model_dir,
    _attn_implementation=args.attn_implementation,
    trust_remote_code=True,
    use_safetensors=True,
    torch_dtype=dtype,
    local_files_only=True,
)
model = model.eval().cuda().to(dtype)

try:
    returned = model.infer(
        tokenizer,
        prompt=prompt,
        image_file=str(image_path),
        output_path=str(output_dir),
        base_size=args.base_size,
        image_size=args.image_size,
        crop_mode=args.crop_mode,
        save_results=True,
    )
    markdown = read_result(output_dir, returned)
    payload = {
        "markdown": markdown,
        "text": markdown,
        "output_dir": str(output_dir),
        "mode": args.mode,
        "model_dir": str(model_dir),
        "image": str(image_path),
    }
    print(f"{RESULT_PREFIX}{json.dumps(payload, ensure_ascii=False)}", flush=True)
finally:
    del model
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

if __name__ == "__main__":
    main()

```

src/ds_ocr_runner/__init__.py

```

"""DS-OCR2 runner service."""

```

src/ds_ocr_runner/api.py

```

from pathlib import Path

from fastapi import APIRouter, Depends, File, HTTPException, UploadFile, status
from sqlalchemy.orm import Session

```

```

from ds_ocr_runner.config import Settings, get_settings
from ds_ocr_runner.db import get_db
from ds_ocr_runner.models import FileRecord, OcrTask, new_id
from ds_ocr_runner.schemas import (
    ClaimTaskRequest,
    CompleteTaskRequest,
    CreateTaskRequest,
    FailTaskRequest,
    FileOut,
    ResultOut,
    TaskOut,
)
from ds_ocr_runner.services import (
    cancel_task,
    claim_next_task,
    complete_task,
    create_file_record,
    create_task,
    fail_task,
)
from ds_ocr_runner.storage import LocalStorage

router = APIRouter()

def get_storage(settings: Settings = Depends(get_settings)) -> LocalStorage:
    return LocalStorage(settings.resolved_storage_root)

@router.get("/healthz")
def healthz(settings: Settings = Depends(get_settings)) -> dict[str, str]:
    return {"status": "ok", "app": settings.app_name, "env": settings.app_env}

@router.post("/api/files", response_model=FileOut, status_code=status.HTTP_201_CREATED)
def upload_file(
    upload: UploadFile = File(...),
    db: Session = Depends(get_db),
    storage: LocalStorage = Depends(get_storage),
) -> FileRecord:
    file_id = new_id("file")
    storage_path, size_bytes, sha256 = storage.save_upload(upload, file_id=file_id)
    return create_file_record(
        db,
        file_id=file_id,
        original_name=upload.filename or Path(storage_path).name,
        storage_path=storage_path,
        content_type=upload.content_type,
        size_bytes=size_bytes,
        sha256=sha256,
    )

@router.get("/api/files/{file_id}", response_model=FileOut)
def get_file(file_id: str, db: Session = Depends(get_db)) -> FileRecord:
    record = db.get(FileRecord, file_id)
    if record is None:
        raise HTTPException(status_code=404, detail="file not found")
    return record

@router.post("/api/ocr/tasks", response_model=TaskOut, status_code=status.HTTP_201_CREATED)
def create_ocr_task(payload: CreateTaskRequest, db: Session = Depends(get_db)) -> OcrTask:
    try:
        return create_task(
            db,
            file_id=payload.file_id,
            mode=payload.mode,
            options=payload.options,
            priority=payload.priority,

```

```

    )
    except KeyError as exc:
        raise HTTPException(status_code=404, detail="file not found") from exc

@router.get("/api/ocr/tasks/{task_id}", response_model=TaskOut)
def get_task(task_id: str, db: Session = Depends(get_db)) -> OcrTask:
    task = db.get(OcrTask, task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="task not found")
    return task

@router.post("/api/ocr/tasks/{task_id}/cancel", response_model=TaskOut)
def cancel_ocr_task(task_id: str, db: Session = Depends(get_db)) -> OcrTask:
    try:
        return cancel_task(db, task_id=task_id)
    except KeyError as exc:
        raise HTTPException(status_code=404, detail="task not found") from exc

@router.get("/api/ocr/tasks/{task_id}/result", response_model=ResultOut)
def get_task_result(task_id: str, db: Session = Depends(get_db)) -> ResultOut:
    task = db.get(OcrTask, task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="task not found")
    if task.result is None:
        raise HTTPException(status_code=404, detail="result not found")
    return task.result

@router.post("/internal/queue/claim", response_model=TaskOut | None)
def claim_task(
    payload: ClaimTaskRequest,
    db: Session = Depends(get_db),
    settings: Settings = Depends(get_settings),
) -> OcrTask | None:
    return claim_next_task(db, worker_id=payload.worker_id, settings=settings)

@router.post("/internal/queue/{task_id}/complete", response_model=TaskOut)
def complete_claimed_task(
    task_id: str,
    payload: CompleteTaskRequest,
    db: Session = Depends(get_db),
) -> OcrTask:
    try:
        return complete_task(
            db,
            task_id=task_id,
            text=payload.text,
            markdown=payload.markdown,
            layout=payload.layout,
            metadata=payload.metadata,
        )
    except KeyError as exc:
        raise HTTPException(status_code=404, detail="task not found") from exc

@router.post("/internal/queue/{task_id}/fail", response_model=TaskOut)
def fail_claimed_task(
    task_id: str,
    payload: FailTaskRequest,
    db: Session = Depends(get_db),
) -> OcrTask:
    try:
        return fail_task(
            db,
            task_id=task_id,
            error_message=payload.error_message,
            retryable=payload.retryable,

```

```
)
except KeyError as exc:
    raise HTTPException(status_code=404, detail="task not found") from exc
```

src/ds_ocr_runner/config.py

```
from functools import lru_cache
from pathlib import Path
from urllib.parse import quote_plus

from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = "DS-OCR2-3090-Runner"
    app_env: str = "local"
    database_url: str | None = None
    postgres_host: str = "localhost"
    postgres_port: int = 5432
    postgres_db: str = "dsocr"
    postgres_user: str = "dsocr"
    postgres_password: str = "dsocr"
    storage_root: Path = Field(default=Path("./storage"))
    ocr_backend: str = "stub"
    deepseek_ocr_python: str = "python"
    deepseek_ocr_model_dir: Path = Field(default=Path("./models/DeepSeek-OCR-2"))
    deepseek_ocr_output_root: Path = Field(default=Path("./storage/results/deepseek-ocr-2"))
    deepseek_ocr_timeout_seconds: int = 900
    deepseek_ocr_base_size: int = 1024
    deepseek_ocr_image_size: int = 768
    deepseek_ocr_crop_mode: bool = True
    deepseek_ocr_attn_implementation: str = "eager"
    deepseek_ocr_dtype: str = "bfloat16"
    queue_visibility_timeout_seconds: int = 900
    worker_id: str = "local-worker"

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    @property
    def resolved_storage_root(self) -> Path:
        return self.storage_root.expanduser().resolve()

    @property
    def resolved_deepseek_ocr_model_dir(self) -> Path:
        return self.deepseek_ocr_model_dir.expanduser().resolve()

    @property
    def resolved_deepseek_ocr_output_root(self) -> Path:
        return self.deepseek_ocr_output_root.expanduser().resolve()

    @property
    def sqlalchemy_database_url(self) -> str:
        if self.database_url:
            return self.database_url

        user = quote_plus(self.postgres_user)
        password = quote_plus(self.postgres_password)
        db_name = quote_plus(self.postgres_db)
        return (
            f"postgresql+psycopg://{user}:{password}"
            f"@{self.postgres_host}:{self.postgres_port}/{db_name}"
        )

    @lru_cache
    def get_settings() -> Settings:
        return Settings()
```

src/ds_ocr_runner/db.py

```
from collections.abc import Generator

from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

from ds_ocr_runner.config import get_settings

class Base(DeclarativeBase):
    pass

engine = create_engine(get_settings().sqlalchemy_database_url, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False, expire_on_commit=False)

def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def create_all() -> None:
    from ds_ocr_runner import models # noqa: F401

    Base.metadata.create_all(bind=engine)
```

src/ds_ocr_runner/main.py

```
from fastapi import FastAPI

from ds_ocr_runner.api import router
from ds_ocr_runner.config import get_settings

def create_app() -> FastAPI:
    settings = get_settings()
    app = FastAPI(title=settings.app_name)
    app.include_router(router)
    return app

app = create_app()
```

src/ds_ocr_runner/manage.py

```
import argparse

from ds_ocr_runner.db import create_all

def main() -> None:
    parser = argparse.ArgumentParser(description="DS-OCR2 runner management commands")
    subparsers = parser.add_subparsers(dest="command", required=True)
    subparsers.add_parser("init-db", help="Create database tables")
    args = parser.parse_args()

    if args.command == "init-db":
        create_all()
        print("database tables created")
```

```
if __name__ == "__main__":
    main()
```

src/ds_ocr_runner/models.py

```
from __future__ import annotations

import enum
import uuid
from datetime import UTC, datetime

from sqlalchemy import DateTime, Enum, ForeignKey, Index, Integer, JSON, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from ds_ocr_runner.db import Base

def new_id(prefix: str) -> str:
    return f"{prefix}_{uuid.uuid4().hex}"

def utc_now() -> datetime:
    return datetime.now(UTC)

class TaskStatus(str, enum.Enum):
    queued = "queued"
    running = "running"
    succeeded = "succeeded"
    failed = "failed"
    canceled = "canceled"

class FileRecord(Base):
    __tablename__ = "ocr_files"

    id: Mapped[str] = mapped_column(String(64), primary_key=True, default=lambda: new_id("file"))
    original_name: Mapped[str] = mapped_column(String(255), nullable=False)
    storage_path: Mapped[str] = mapped_column(String(1024), nullable=False)
    content_type: Mapped[str | None] = mapped_column(String(255))
    size_bytes: Mapped[int] = mapped_column(Integer, nullable=False)
    sha256: Mapped[str] = mapped_column(String(64), nullable=False, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utc_now)

    tasks: Mapped[list[OcrTask]] = relationship(back_populates="file")

class OcrTask(Base):
    __tablename__ = "ocr_tasks"

    id: Mapped[str] = mapped_column(String(64), primary_key=True, default=lambda: new_id("task"))
    file_id: Mapped[str] = mapped_column(ForeignKey("ocr_files.id"), nullable=False, index=True)
    mode: Mapped[str] = mapped_column(String(64), nullable=False, default="document_markdown")
    status: Mapped[TaskStatus] = mapped_column(
        Enum(TaskStatus, name="task_status"),
        nullable=False,
        default=TaskStatus.queued,
        index=True,
    )
    options: Mapped[dict] = mapped_column(JSON, default=dict)
    priority: Mapped[int] = mapped_column(Integer, nullable=False, default=100)
    attempts: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    max_attempts: Mapped[int] = mapped_column(Integer, nullable=False, default=3)
    worker_id: Mapped[str | None] = mapped_column(String(128))
    locked_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    started_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    completed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    error_message: Mapped[str | None] = mapped_column(Text)
```

```

created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utc_now, index=True)
updated_at: Mapped[datetime] = mapped_column(
    DateTime(timezone=True),
    default=utc_now,
    onupdate=utc_now,
)

file: Mapped[FileRecord] = relationship(back_populates="tasks")
result: Mapped[OcrResult | None] = relationship(back_populates="task", uselist=False)

class OcrResult(Base):
    __tablename__ = "ocr_results"

    id: Mapped[str] = mapped_column(String(64), primary_key=True, default=lambda: new_id("result"))
    task_id: Mapped[str] = mapped_column(ForeignKey("ocr_tasks.id"), unique=True, nullable=False)
    text: Mapped[str] = mapped_column(Text, default="")
    markdown: Mapped[str] = mapped_column(Text, default="")
    layout: Mapped[dict] = mapped_column(JSON, default=dict)
    metadata_json: Mapped[dict] = mapped_column(JSON, default=dict)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), default=utc_now)

    task: Mapped[OcrTask] = relationship(back_populates="result")

Index(
    "ix_ocr_tasks_queue_claim",
    OcrTask.status,
    OcrTask.priority,
    OcrTask.created_at,
)

```

src/ds_ocr_runner/ocr_backend.py

```

from dataclasses import dataclass
import json
from pathlib import Path
import subprocess
from typing import Protocol

from ds_ocr_runner.config import get_settings
from ds_ocr_runner.models import OcrTask

@dataclass(frozen=True)
class OcrOutput:
    text: str
    markdown: str
    layout: dict
    metadata: dict

class OcrBackend(Protocol):
    def infer(self, *, task: OcrTask, input_path: Path) -> OcrOutput:
        ...

class StubOcrBackend:
    def infer(self, *, task: OcrTask, input_path: Path) -> OcrOutput:
        message = (
            "OCR inference is disabled because OCR_BACKEND=stub. "
            "Configure a DeepSeek-OCR-2 backend on a GPU host before production use."
        )
        return OcrOutput(
            text=message,
            markdown=f"# OCR Stub Result\n\n{message}\n\nInput: `{input_path.name}`\n",
            layout={},
            metadata={"backend": "stub", "mode": task.mode},

```

```

)

class ExternalDeepSeekOcr2Backend:
    result_prefix = "__DS_OCR_RESULT__="

    def infer(self, *, task: OcrTask, input_path: Path) -> OcrOutput:
        settings = get_settings()
        output_dir = settings.resolved_deepseek_ocr_output_root / task.id
        output_dir.mkdir(parents=True, exist_ok=True)

        cmd = [
            settings.deepseek_ocr_python,
            "scripts/run_deepseek_ocr2.py",
            "--model-dir",
            str(settings.resolved_deepseek_ocr_model_dir),
            "--image",
            str(input_path),
            "--output-dir",
            str(output_dir),
            "--mode",
            task.mode,
            "--base-size",
            str(settings.deepseek_ocr_base_size),
            "--image-size",
            str(settings.deepseek_ocr_image_size),
            "--attn-implementation",
            settings.deepseek_ocr_attn_implementation,
            "--dtype",
            settings.deepseek_ocr_dtype,
        ]
        cmd.append("--crop-mode" if settings.deepseek_ocr_crop_mode else "--no-crop-mode")

        completed = subprocess.run(
            cmd,
            cwd=Path(__file__).resolve().parents[2],
            text=True,
            capture_output=True,
            timeout=settings.deepseek_ocr_timeout_seconds,
            check=False,
        )
        if completed.returncode != 0:
            raise RuntimeError(
                "DeepSeek-OCR-2 inference failed\n"
                f"stdout:\n{completed.stdout}\n"
                f"stderr:\n{completed.stderr}"
            )

        payload = None
        for line in reversed(completed.stdout.splitlines()):
            if line.startswith(self.result_prefix):
                payload = json.loads(line.removeprefix(self.result_prefix))
                break
        if payload is None:
            raise RuntimeError(
                "DeepSeek-OCR-2 inference did not emit a result payload\n"
                f"stdout:\n{completed.stdout}\n"
                f"stderr:\n{completed.stderr}"
            )

        markdown = payload.get("markdown", "")
        return OcrOutput(
            text=payload.get("text", markdown),
            markdown=markdown,
            layout={},
            metadata={
                "backend": "deepseek-ocr-2",
                "mode": task.mode,
                "output_dir": payload.get("output_dir"),
                "model_dir": payload.get("model_dir"),
            },
        ),

```

```

    )

def build_ocr_backend(name: str) -> OcrBackend:
    normalized = name.strip().lower()
    if normalized == "stub":
        return StubOcrBackend()
    if normalized in {"deepseek", "deepseek-ocr-2", "deepseek_ocr_2"}:
        return ExternalDeepSeekOcr2Backend()
    raise ValueError(f"Unsupported OCR_BACKEND: {name}")

```

src/ds_ocr_runner/schemas.py

```

from datetime import datetime
from typing import Any

from pydantic import BaseModel, ConfigDict, Field

from ds_ocr_runner.models import TaskStatus

class FileOut(BaseModel):
    id: str
    original_name: str
    content_type: str | None
    size_bytes: int
    sha256: str
    created_at: datetime

    model_config = ConfigDict(from_attributes=True)

class CreateTaskRequest(BaseModel):
    file_id: str
    mode: str = Field(default="document_markdown", max_length=64)
    options: dict[str, Any] = Field(default_factory=dict)
    priority: int = Field(default=100, ge=0, le=1000)

class TaskOut(BaseModel):
    id: str
    file_id: str
    mode: str
    status: TaskStatus
    options: dict[str, Any]
    priority: int
    attempts: int
    max_attempts: int
    worker_id: str | None
    error_message: str | None
    created_at: datetime
    updated_at: datetime
    started_at: datetime | None
    completed_at: datetime | None

    model_config = ConfigDict(from_attributes=True)

class ResultOut(BaseModel):
    id: str
    task_id: str
    text: str
    markdown: str
    layout: dict[str, Any]
    metadata_json: dict[str, Any]
    created_at: datetime

    model_config = ConfigDict(from_attributes=True)

```

```

class ClaimTaskRequest(BaseModel):
    worker_id: str = Field(min_length=1, max_length=128)

class CompleteTaskRequest(BaseModel):
    text: str = ""
    markdown: str = ""
    layout: dict[str, Any] = Field(default_factory=dict)
    metadata: dict[str, Any] = Field(default_factory=dict)

class FailTaskRequest(BaseModel):
    error_message: str
    retryable: bool = True

```

src/ds_ocr_runner/services.py

```

from datetime import UTC, datetime, timedelta

from sqlalchemy import or_, select
from sqlalchemy.orm import Session

from ds_ocr_runner.config import Settings
from ds_ocr_runner.models import FileRecord, OcrResult, OcrTask, TaskStatus, utc_now

def create_file_record(
    db: Session,
    *,
    file_id: str,
    original_name: str,
    storage_path: str,
    content_type: str | None,
    size_bytes: int,
    sha256: str,
) -> FileRecord:
    record = FileRecord(
        id=file_id,
        original_name=original_name,
        storage_path=storage_path,
        content_type=content_type,
        size_bytes=size_bytes,
        sha256=sha256,
    )
    db.add(record)
    db.commit()
    db.refresh(record)
    return record

def create_task(
    db: Session,
    *,
    file_id: str,
    mode: str,
    options: dict,
    priority: int,
) -> OcrTask:
    file_record = db.get(FileRecord, file_id)
    if file_record is None:
        raise KeyError(file_id)

    task = OcrTask(file_id=file_id, mode=mode, options=options, priority=priority)
    db.add(task)
    db.commit()
    db.refresh(task)
    return task

```

```

def claim_next_task(db: Session, *, worker_id: str, settings: Settings) -> OcrTask | None:
    now = utc_now()
    stale_before = now - timedelta(seconds=settings.queue_visibility_timeout_seconds)

    stmt = (
        select(OcrTask)
        .where(
            OcrTask.status.in_([TaskStatus.queued, TaskStatus.running]),
            OcrTask.attempts < OcrTask.max_attempts,
            or_(OcrTask.status == TaskStatus.queued, OcrTask.locked_at < stale_before),
        )
        .order_by(OcrTask.priority.asc(), OcrTask.created_at.asc())
        .with_for_update(skip_locked=True)
        .limit(1)
    )

    with db.begin():
        task = db.execute(stmt).scalar_one_or_none()
        if task is None:
            return None
        task.status = TaskStatus.running
        task.worker_id = worker_id
        task.locked_at = now
        task.started_at = task.started_at or now
        task.attempts += 1
        db.add(task)

    db.refresh(task)
    return task

def complete_task(
    db: Session,
    *,
    task_id: str,
    text: str,
    markdown: str,
    layout: dict,
    metadata: dict,
) -> OcrTask:
    task = db.get(OcrTask, task_id)
    if task is None:
        raise KeyError(task_id)
    if task.status == TaskStatus.canceled:
        return task

    result = task.result or OcrResult(task_id=task.id)
    result.text = text
    result.markdown = markdown
    result.layout = layout
    result.metadata_json = metadata

    task.status = TaskStatus.succeeded
    task.completed_at = datetime.now(UTC)
    task.error_message = None

    db.add(result)
    db.add(task)
    db.commit()
    db.refresh(task)
    return task

def fail_task(db: Session, *, task_id: str, error_message: str, retryable: bool) -> OcrTask:
    task = db.get(OcrTask, task_id)
    if task is None:
        raise KeyError(task_id)
    if task.status == TaskStatus.canceled:
        return task

```

```

def forward(
    self,
    input_ids: torch.LongTensor = None,
    attention_mask: Optional[torch.Tensor] = None,
    position_ids: Optional[torch.LongTensor] = None,
    past_key_values: Optional[List[torch.FloatTensor]] = None,
    inputs_embeds: Optional[torch.FloatTensor] = None,
    use_cache: Optional[bool] = None,
    output_attentions: Optional[bool] = None,
    output_hidden_states: Optional[bool] = None,
    images: Optional[torch.FloatTensor] = None,
    images_seq_mask: Optional[torch.FloatTensor] = None,
    images_spatial_crop: Optional[torch.FloatTensor] = None,
    return_dict: Optional[bool] = None,
) -> Union[Tuple, BaseModelOutputWithPast]:

```

```

    if inputs_embeds is None:
        # inputs_embeds = self.embed_tokens(input_ids)
        inputs_embeds = self.get_input_embeddings()(input_ids)

```

```

    sam_model = getattr(self, 'sam_model', None)
    # sam_model = self.sam_model
    qwen2_model = getattr(self, 'qwen2_model', None)

```

```

    if sam_model is not None and (input_ids.shape[1] != 1 or self.training) and torch.sum(images[0][1]).item()
!= 0:

```

```

        idx = 0

```

```

        # sam_model = torch.jit.script(sam_model)

```

```

        # start_time = time.time()
        for image, crop_shape in zip(images, images_spatial_crop):
            images_in_this_batch = []

```

```

            patches = image[0]
            image_ori = image[1]

```

```

            with torch.no_grad():
                # with torch.inference_mode():

```

```

                if torch.sum(patches).item() != 0:
                    # P, C, H, W = patches.shape
                    crop_flag = 1
                    local_features_1 = sam_model(patches)

                    local_features_2 = qwen2_model(local_features_1)
                    # vit_time = time.time()
                    local_features = local_features_2
                    local_features = self.projector(local_features)

                    global_features_1 = sam_model(image_ori)
                    global_features_2 = qwen2_model(global_features_1)
                    global_features = global_features_2
                    global_features = self.projector(global_features)

```

```

                    print('=====')
                    print('BASE: ', global_features.shape)
                    print('PATCHES: ', local_features.shape)

```

```

print('=====')

_, hw, n_dim = global_features.shape
# h = w = int(hw ** 0.5)

_, hw2, n_dim2 = local_features.shape
# h2 = w2 = int(hw2 ** 0.5)

global_features = global_features.view(-1, n_dim)

local_features = local_features.view(-1, n_dim2)

global_local_features = torch.cat([local_features, global_features,
self.view_seperator[None, :]], dim=0)

# end_time = time.time()

# print('sam: ', sam_time - start_time)
# print('vit: ', vit_time - sam_time)
# print('all: ', end_time - start_time)

# exit()

else:
    global_features_1 = sam_model(image_ori)
    global_features_2 = qwen2_model(global_features_1)
    global_features = global_features_2
    global_features = self.projector(global_features)
    print('=====')
    print('BASE: ', global_features.shape)
    print('NO PATCHES')
    print('=====')
    _, hw, n_dim = global_features.shape
    # h = w = int(hw ** 0.5)

    # global_features = global_features.view(h, w, n_dim)

    # global_features = torch.cat(
    #     [global_features, self.image_newline[None, None, :].expand(h, 1, n_dim)], dim=1
    # )

    global_features = global_features.view(-1, n_dim)

    global_local_features = torch.cat([global_features, self.view_seperator[None, :]], dim=0)

    images_in_this_batch.append(global_local_features)

# print(inputs_embeds.shape)

if images_in_this_batch:
    images_in_this_batch = torch.cat(images_in_this_batch, dim=0)
    # exit()

    inputs_embeds[idx].masked_scatter_(images_seq_mask[idx].unsqueeze(-1).cuda(),
images_in_this_batch)

    idx += 1

return super(DeepseekOCR2Model, self).forward(
    input_ids=None, attention_mask=attention_mask, past_key_values=past_key_values,
    inputs_embeds=inputs_embeds, use_cache=use_cache, position_ids = position_ids,
    output_attentions=output_attentions, output_hidden_states=output_hidden_states,
    return_dict=return_dict
)

```

```

class Deepseek0CR2ForCausalLM(DeepseekV2ForCausalLM):

    config_class = Deepseek0CR2Config
    # supports_gradient_checkpointing = True

    def __init__(self, config):
        super(DeepseekV2ForCausalLM, self).__init__(config)
        self.model = Deepseek0CR2Model(config)

        self.vocab_size = config.vocab_size

        self.lm_head = nn.Linear(config.hidden_size, config.vocab_size, bias=False)

        # self.lm_head = nn.Linear(config.hidden_size, config.vocab_size, bias=False)

        # Initialize weights and apply final processing
        self.post_init()

    def get_model(self):
        return self.model

    def forward(
        self,
        input_ids: torch.LongTensor = None,
        attention_mask: Optional[torch.Tensor] = None,
        position_ids: Optional[torch.LongTensor] = None,
        past_key_values: Optional[List[torch.FloatTensor]] = None,
        inputs_embeds: Optional[torch.FloatTensor] = None,
        labels: Optional[torch.LongTensor] = None,
        use_cache: Optional[bool] = None,
        output_attentions: Optional[bool] = None,
        output_hidden_states: Optional[bool] = None,
        images: Optional[torch.FloatTensor] = None,
        images_seq_mask: Optional[torch.FloatTensor] = None,
        images_spatial_crop: Optional[torch.FloatTensor] = None,
        return_dict: Optional[bool] = None,
    ) -> Union[Tuple, CausalLMOutputWithPast]:
        output_attentions = output_attentions if output_attentions is not None else self.config.output_attentions
        output_hidden_states = (
            output_hidden_states if output_hidden_states is not None else self.config.output_hidden_states
        )
        return_dict = return_dict if return_dict is not None else self.config.use_return_dict

        outputs = self.model(
            input_ids=input_ids,
            past_key_values=past_key_values,
            attention_mask=attention_mask,
            position_ids=position_ids,
            inputs_embeds=inputs_embeds,
            use_cache=use_cache,
            output_attentions=output_attentions,
            output_hidden_states=output_hidden_states,
            images=images,
            images_seq_mask = images_seq_mask,
            images_spatial_crop = images_spatial_crop,
            return_dict=return_dict
        )

        # print(transformer_outputs)

        hidden_states = outputs[0]
        logits = self.lm_head(hidden_states)
        logits = logits.float()

```

```

# logits

loss = None
if labels is not None:
    # Shift so that tokens < n predict n
    shift_logits = logits[..., :-1, :].contiguous()
    shift_labels = labels[..., 1:].contiguous()
    # Flatten the tokens
    loss_fct = CrossEntropyLoss()
    shift_logits = shift_logits.view(-1, self.config.vocab_size)
    shift_labels = shift_labels.view(-1)
    # Enable model parallelism
    shift_labels = shift_labels.to(shift_logits.device)
    loss = loss_fct(shift_logits, shift_labels)

if not return_dict:
    output = (logits,) + outputs[1:]
    return (loss,) + output if loss is not None else output

return CausalLMOutputWithPast(
    loss=loss,
    logits=logits,
    past_key_values=outputs.past_key_values,
    hidden_states=outputs.hidden_states,
    attentions=outputs.attentions,
)

```

```

def prepare_inputs_for_generation(
    self, input_ids, past_key_values=None, attention_mask=None, inputs_embeds=None, **kwargs
):

```

```

    # Omit tokens covered by past_key_values
    past_length = 0
    if past_key_values is not None:
        if isinstance(past_key_values, Cache):
            cache_length = past_key_values.get_seq_length()
            past_length = past_key_values.seen_tokens
            max_cache_length = past_key_values.get_max_length()
        else:
            cache_length = past_length = past_key_values[0][0].shape[2]
            max_cache_length = None

```

```

    # Keep only the unprocessed tokens:

```

```

    # 1 - If the length of the attention_mask exceeds the length of input_ids, then we are in a setting

```

where

```

    # some of the inputs are exclusively passed as part of the cache (e.g. when passing input_embeds as
    # input)

```

```

    if attention_mask is not None and attention_mask.shape[1] > input_ids.shape[1]:

```

```

        input_ids = input_ids[:, -(attention_mask.shape[1] - past_length) :]

```

```

    # 2 - If the past_length is smaller than input_ids', then input_ids holds all input tokens. We can

```

discard

```

    # input_ids based on the past_length.

```

```

    elif past_length < input_ids.shape[1]:

```

```

        input_ids = input_ids[:, past_length:]

```

```

    # 3 - Otherwise (past_length >= input_ids.shape[1]), let's assume input_ids only has unprocessed

```

tokens.

```

    # If we are about to go beyond the maximum cache length, we need to crop the input attention mask.

```

```

    if (

```

```

        max_cache_length is not None

```

```

        and attention_mask is not None

```

```

        and cache_length + input_ids.shape[1] > max_cache_length

```

```

    ):

```

```

        attention_mask = attention_mask[:, -max_cache_length:]

```

```

position_ids = kwargs.get("position_ids", None)

```

```

if attention_mask is not None and position_ids is None:

```

```

    # create position_ids on the fly for batch generation

```

```

    position_ids = attention_mask.long().cumsum(-1) - 1

```

```

    position_ids.masked_fill_(attention_mask == 0, 1)

```

```

    if past_key_values:

```

```

        position_ids = position_ids[:, -input_ids.shape[1] :]

# if self.generation_config.cache_implementation == "static":
#     # generation with static cache
#     cache_position = kwargs.get("cache_position", None)
#     if cache_position is None:
#         past_length = 0
#     else:
#         past_length = cache_position[-1] + 1
#     input_ids = input_ids[:, past_length:]
#     position_ids = position_ids[:, past_length:]

# TODO @gante we should only keep a `cache_position` in generate, and do +=1.
# same goes for position ids. Could also help with continued generation.
cache_position = torch.arange(past_length, past_length + position_ids.shape[-1],
device=position_ids.device)

# if `inputs_embeds` are passed, we only want to use them in the 1st generation step
if inputs_embeds is not None and past_key_values is None:
    model_inputs = {"inputs_embeds": inputs_embeds}
else:
    model_inputs = {"input_ids": input_ids}

model_inputs.update(
    {
        "position_ids": position_ids,
        "past_key_values": past_key_values,
        "use_cache": kwargs.get("use_cache"),
        "attention_mask": attention_mask,
        "images": kwargs.get("images", None),
        "images_seq_mask": kwargs.get("images_seq_mask", None),
        "images_spatial_crop": kwargs.get("images_spatial_crop", None),
    }
)
return model_inputs

def disable_torch_init(self):
    """
    Disable the redundant torch default initialization to accelerate model creation.
    """
    import torch
    setattr(torch.nn.Linear, "reset_parameters", lambda self: None)
    setattr(torch.nn.LayerNorm, "reset_parameters", lambda self: None)

def infer(self, tokenizer, prompt='', image_file='', output_path = '', base_size=1024, image_size=640,
crop_mode=True, test_compress=False, save_results=False, eval_mode=False):
    self.disable_torch_init()

    os.makedirs(output_path, exist_ok=True)
    os.makedirs(f'{output_path}/images', exist_ok=True)

    if prompt and image_file:
        conversation = [
            {
                "role": "<|User|>",
                # "content": "<image>\n<|grounding|>Given the layout of the image. ",
                "content": f'{prompt}',
                # "content": "君不见黄河之水天上来的下一句是什么? ",
                # "content": "<image>\nFree OCR. ",
                # "content": "<image>\nParse the figure. ",
                # "content": "<image>\nExtract the text in the image. ",
                "images": [f'{image_file}'],
            },
            {"role": "<|Assistant|>", "content": ""},
        ]

    elif prompt:
        conversation = [

```

```

        {
            "role": "<|User|>",
            # "content": "<image>\n<|grounding|>Given the layout of the image. ",
            "content": f'{prompt}',
            # "content": "君不见黄河之水天上来的下一句是什么？ ",
            # "content": "<image>\nFree OCR. ",
            # "content": "<image>\nParse the figure. ",
            # "content": "<image>\nExtract the text in the image. ",
            # "images": [f'{image_file}'],
        },
        {"role": "<|Assistant|>", "content": ""},
    ]
else:
    assert False, f'prompt is none!'

prompt = format_messages(conversations=conversation, sft_format='plain', system_prompt='')

patch_size = 16
downsample_ratio = 4
images = load_pil_images(conversation)

valid_img_tokens = 0
ratio = 1

image_draw = images[0].copy()

w,h = image_draw.size
# print(w, h)
ratio = 1 - ((max(w, h) - min(w, h)) / (max(w, h)))

image_transform=BasicImageTransform(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5), normalize=True)
images_seq_mask = []

image_token = '<image>'
image_token_id = 128815
text_splits = prompt.split(image_token)

images_list, images_crop_list, images_seq_mask = [], [], []
tokenized_str = []
images_spatial_crop = []
for text_sep, image in zip(text_splits, images):

    tokenized_sep = text_encode(tokenizer, text_sep, bos=False, eos=False)
    tokenized_str += tokenized_sep
    images_seq_mask += [False] * len(tokenized_sep)

    if crop_mode:

        if image.size[0] <= 768 and image.size[1] <= 768:
            crop_ratio = [1, 1]

        else:
            if crop_mode:
                # best_width, best_height = select_best_resolution(image.size, self.candidate_resolutions)
                images_crop_raw, crop_ratio = dynamic_preprocess(image)
            else:
                # best_width, best_height = self.image_size, self.image_size
                crop_ratio = [1, 1]

        """process the global view"""
        # image = image.resize((base_size, base_size))
        global_view = ImageOps.pad(image, (base_size, base_size),
                                    color=tuple(int(x * 255) for x in image_transform.mean))

        if base_size == 1024:
            valid_img_tokens += int(256 * ratio)
        elif base_size == 1280:
            valid_img_tokens += int(400 * ratio)
        # elif base_size == 640:
        #     valid_img_tokens += int(100 * ratio)

```

```

images_list.append(image_transform(global_view).to(torch.bfloat16))

# global_view_tensor = image_transform(global_view).to(torch.bfloat16)

width_crop_num, height_crop_num = crop_ratio

images_spatial_crop.append([width_crop_num, height_crop_num])

if width_crop_num > 1 or height_crop_num > 1:
    """process the local views"""

    for i in range(len(images_crop_raw)):
        images_crop_list.append(image_transform(images_crop_raw[i]).to(torch.bfloat16))

if image_size == 768:
    valid_img_tokens += len(images_crop_list) * 144

num_queries = math.ceil((image_size // patch_size) / downsample_ratio)
num_queries_base = math.ceil((base_size // patch_size) / downsample_ratio)

"""add image tokens"""

tokenized_image = ([image_token_id] * num_queries_base) * num_queries_base
tokenized_image += [image_token_id]
if width_crop_num > 1 or height_crop_num > 1:
    tokenized_image += ([image_token_id] * (num_queries * width_crop_num)) * (
        num_queries * height_crop_num)
tokenized_str += tokenized_image
images_seq_mask += [True] * len(tokenized_image)
# num_image_tokens.append(len(tokenized_image))

else:
    # best_width, best_height = self.image_size, self.image_size
    # print(image.size, (best_width, best_height)) # check the select_best_resolutions func

    """process the global view"""
    if image_size <= 768:
        print('directly resize')
        image = image.resize((image_size, image_size))
    # else:
    global_view = ImageOps.pad(image, (image_size, image_size),
        color=tuple(int(x * 255) for x in image_transform.mean))
    images_list.append(image_transform(global_view).to(torch.bfloat16))

    if base_size == 1024:
        valid_img_tokens += int(256 * ratio)
    elif base_size == 1280:
        valid_img_tokens += int(400 * ratio)
    elif base_size == 640:
        valid_img_tokens += int(100 * 1)
    elif base_size == 512:
        valid_img_tokens += int(64 * 1)
    elif base_size == 768:
        valid_img_tokens += int(144 * 1)

    width_crop_num, height_crop_num = 1, 1

    images_spatial_crop.append([width_crop_num, height_crop_num])

    """add image tokens"""

```

```

        num_queries = math.ceil((image_size // patch_size) / downsample_ratio)

        tokenized_image = ([image_token_id] * num_queries) * num_queries
        tokenized_image += [image_token_id]
        # tokenized_image += ([self.image_token_id] * (num_queries * width_crop_num) +
[self.image_token_id]) * (
        #         num_queries * height_crop_num)
        tokenized_str += tokenized_image
        images_seq_mask += [True] * len(tokenized_image)
        # num_image_tokens.append(len(tokenized_image))

    """process the last text split"""
    tokenized_sep = text_encode(tokenizer, text_splits[-1], bos=False, eos=False)
    tokenized_str += tokenized_sep
    images_seq_mask += [False] * len(tokenized_sep)

    """add the bos tokens"""
    bos_id = 0
    tokenized_str = [bos_id] + tokenized_str
    images_seq_mask = [False] + images_seq_mask

    input_ids = torch.LongTensor(tokenized_str)

    images_seq_mask = torch.tensor(images_seq_mask, dtype=torch.bool)

    if len(images_list) == 0:
        images_ori = torch.zeros((1, 3, image_size, image_size))
        images_spatial_crop = torch.zeros((1, 2), dtype=torch.long)
        images_crop = torch.zeros((1, 3, base_size, base_size))
    else:
        images_ori = torch.stack(images_list, dim=0)
        images_spatial_crop = torch.tensor(images_spatial_crop, dtype=torch.long)
        if images_crop_list:
            images_crop = torch.stack(images_crop_list, dim=0)
        else:
            images_crop = torch.zeros((1, 3, base_size, base_size))

    if not eval_mode:
        streamer = NoEOSTextStreamer(tokenizer, skip_prompt=True, skip_special_tokens=False)
        with torch.autocast("cuda", dtype=torch.bfloat16):
            with torch.no_grad():
                output_ids = self.generate(
                    input_ids.unsqueeze(0).cuda(),
                    images=[images_crop.cuda(), images_ori.cuda()],
                    images_seq_mask = images_seq_mask.unsqueeze(0).cuda(),
                    images_spatial_crop = images_spatial_crop,
                    # do_sample=False,
                    # num_beams = 1,
                    temperature=0.0,
                    eos_token_id=tokenizer.eos_token_id,
                    streamer=streamer,
                    max_new_tokens=8192,
                    no_repeat_ngram_size = 20,
                    use_cache = True
                )
    else:
        with torch.autocast("cuda", dtype=torch.bfloat16):
            with torch.no_grad():
                output_ids = self.generate(
                    input_ids.unsqueeze(0).cuda(),

```

```

        images=[(images_crop.cuda(), images_ori.cuda())],
        images_seq_mask = images_seq_mask.unsqueeze(0).cuda(),
        images_spatial_crop = images_spatial_crop,
        # do_sample=False,
        # num_beams = 1,
        temperature=0.0,
        eos_token_id=tokenizer.eos_token_id,
        max_new_tokens=8192,
        no_repeat_ngram_size = 35,
        use_cache = True
    )

if '<image>' in conversation[0]['content'] and eval_mode:
    outputs = tokenizer.decode(output_ids[0, input_ids.unsqueeze(0).cuda().shape[1]:])
    stop_str = '<| end__of__sentence |>'
    if outputs.endswith(stop_str):
        outputs = outputs[:-len(stop_str)]
    # re_match
    outputs = outputs.strip()

    return outputs

if '<image>' in conversation[0]['content'] and test_compress:
    outputs = tokenizer.decode(output_ids[0, input_ids.unsqueeze(0).cuda().shape[1]:])
    pure_texts_outputs_token_length = len(text_encode(tokenizer, outputs, bos=False, eos=False))
    print('='*50)
    print('image size: ', (w, h))
    print('valid image tokens: ', int(valid_img_tokens))
    print('output texts tokens (valid): ', pure_texts_outputs_token_length)
    print('compression ratio: ', round(pure_texts_outputs_token_length/valid_img_tokens, 2))
    print('='*50)

if '<image>' in conversation[0]['content'] and save_results:
    outputs = tokenizer.decode(output_ids[0, input_ids.unsqueeze(0).cuda().shape[1]:])
    stop_str = '<| end__of__sentence |>'

    print('='*15 + 'save results:' + '='*15)

    # # # # conv.messages[-1][-1] = outputs
    if outputs.endswith(stop_str):
        outputs = outputs[:-len(stop_str)]
    outputs = outputs.strip()

    matches_ref, matches_images, mathes_other = re_match(outputs)
    # print(matches_ref)
    result = process_image_with_refs(image_draw, matches_ref, output_path)

    for idx, a_match_image in enumerate(tqdm(matches_images, desc="image")):
        outputs = outputs.replace(a_match_image, '!['](images/' + str(idx) + '.jpg)\n')

    for idx, a_match_other in enumerate(tqdm(mathes_other, desc="other")):
        outputs = outputs.replace(a_match_other, '').replace('\coloneqq', ':=').replace('\eqqcolon',
'=:')

    # if 'structural formula' in conversation[0]['content']:
    #     outputs = '<smiles>' + outputs + '</smiles>'
    with open(f'{output_path}/result.mmd', 'w', encoding = 'utf-8') as afile:
        afile.write(outputs)

    if 'line_type' in outputs:
        import matplotlib.pyplot as plt
        lines = eval(outputs)['Line']['line']

        line_type = eval(outputs)['Line']['line_type']
        # print(lines)

        endpoints = eval(outputs)['Line']['line_endpoint']

```

```

fig, ax = plt.subplots(figsize=(3,3), dpi=200)
ax.set_xlim(-15, 15)
ax.set_ylim(-15, 15)

for idx, line in enumerate(lines):
    try:
        p0 = eval(line.split(' -- ')[0])
        p1 = eval(line.split(' -- ')[-1])

        if line_type[idx] == '--':
            ax.plot([p0[0], p1[0]], [p0[1], p1[1]], linewidth=0.8, color='k')
        else:
            ax.plot([p0[0], p1[0]], [p0[1], p1[1]], linewidth = 0.8, color = 'k')

        ax.scatter(p0[0], p0[1], s=5, color = 'k')
        ax.scatter(p1[0], p1[1], s=5, color = 'k')
    except:
        pass

for endpoint in endpoints:

    label = endpoint.split(': ')[0]
    (x, y) = eval(endpoint.split(': ')[1])
    ax.annotate(label, (x, y), xytext=(1, 1), textcoords='offset points',
                fontsize=5, fontweight='light')

plt.savefig(f'{output_path}/geo.jpg')
plt.close()

result.save(f"{output_path}/result_with_boxes.jpg")

```

models/DeepSeek-OCR-2/modeling_deepseekv2.py

```

# coding=utf-8
# Copyright 2023 DeepSeek-AI and The HuggingFace Inc. team. All rights reserved.
#
# This code is based on EleutherAI's GPT-NeoX Library and the GPT-NeoX
# and OPT implementations in this library. It has been modified from its
# original forms to accommodate minor architectural differences compared
# to GPT-NeoX and OPT used by the Meta AI team that trained the model.
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     ***
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
""" PyTorch DeepSeek model and compatible with both DeepSeekV2 and DeepSeekV3 """
import math
import warnings
from typing import List, Optional, Tuple, Union
import numpy as np

import torch
import torch.nn.functional as F
import torch.nn.utils.checkpoint
import torch.distributed as dist
from einops import repeat
from torch import nn
from torch.nn import BCEWithLogitsLoss, CrossEntropyLoss, MSELoss

from transformers.activations import ACT2FN

```

```

from transformers.cache_utils import Cache, DynamicCache
from transformers.modeling_attn_mask_utils import _prepare_4d_causal_attention_mask
from transformers.models.llama.modeling_llama import (
    LlamaAttention,
    LlamaFlashAttention2
)
from transformers.modeling_outputs import (
    BaseModelOutputWithPast,
    CausalLMOutputWithPast,
    SequenceClassifierOutputWithPast,
)
from transformers.modeling_utils import PreTrainedModel
from transformers.pytorch_utils import (
    ALL_LAYERNORM_LAYERS,
    is_torch_greater_or_equal_than_1_13,
)
from transformers.utils import (
    add_start_docstrings,
    add_start_docstrings_to_model_forward,
    is_flash_attn_2_available,
    is_flash_attn_greater_or_equal_2_10,
    logging,
    replace_return_docstrings,
)
from transformers.utils.import_utils import is_torch_fx_available

from .configuration_deepseek_v2 import DeepseekV2Config

if is_flash_attn_2_available():
    from flash_attn import flash_attn_func, flash_attn_varlen_func
    from flash_attn.bert_padding import index_first_axis, pad_input, unpad_input  # noqa

# This makes `_prepare_4d_causal_attention_mask` a leaf function in the FX graph.
# It means that the function will not be traced through and simply appear as a node in the graph.
if is_torch_fx_available():
    if not is_torch_greater_or_equal_than_1_13:
        import torch.fx

    _prepare_4d_causal_attention_mask = torch.fx.wrap(_prepare_4d_causal_attention_mask)

logger = logging.get_logger(__name__)

_CONFIG_FOR_DOC = "DeepseekV2Config"

def _get_unpad_data(attention_mask):
    seqlens_in_batch = attention_mask.sum(dim=-1, dtype=torch.int32)
    indices = torch.nonzero(attention_mask.flatten(), as_tuple=False).flatten()
    max_seqlen_in_batch = seqlens_in_batch.max().item()
    cu_seqlens = F.pad(
        torch.cumsum(seqlens_in_batch, dim=0, dtype=torch.torch.int32), (1, 0)
    )
    return (
        indices,
        cu_seqlens,
        max_seqlen_in_batch,
    )

class DeepseekV2RMSNorm(nn.Module):
    def __init__(self, hidden_size, eps=1e-6):
        """
        DeepseekV2RMSNorm is equivalent to T5LayerNorm
        """
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size))
        self.variance_epsilon = eps

    def forward(self, hidden_states):
        input_dtype = hidden_states.dtype
        hidden_states = hidden_states.to(torch.float32)

```

```

variance = hidden_states.pow(2).mean(-1, keepdim=True)
hidden_states = hidden_states * torch.rsqrt(variance + self.variance_epsilon)
return self.weight * hidden_states.to(input_dtype)

```

```
ALL_LAYERNORM_LAYERS.append(DeepseekV2RMSNorm)
```

```
class DeepseekV2RotaryEmbedding(nn.Module):
```

```

    def __init__(self, dim, max_position_embeddings=2048, base=10000, device=None):
        super().__init__()

```

```

        self.dim = dim
        self.max_position_embeddings = max_position_embeddings
        self.base = base
        inv_freq = 1.0 / (
            self.base ** (torch.arange(0, self.dim, 2).float().to(device) / self.dim)
        )
        self.register_buffer("inv_freq", inv_freq, persistent=False)

```

```
    # Build here to make `torch.jit.trace` work.
```

```

    self._set_cos_sin_cache(
        seq_len=max_position_embeddings,
        device=self.inv_freq.device,
        dtype=torch.get_default_dtype(),
    )
    self.max_seq_len_cached = None

```

```

    def _set_cos_sin_cache(self, seq_len, device, dtype):
        self.max_seq_len_cached = seq_len
        t = torch.arange(
            self.max_seq_len_cached, device=device, dtype=self.inv_freq.dtype
        )

```

```

        freqs = torch.outer(t, self.inv_freq.to(t.device))
        # Different from paper, but it uses a different permutation in order to obtain the same calculation
        emb = torch.cat((freqs, freqs), dim=-1)
        self.register_buffer("cos_cached", emb.cos().to(dtype), persistent=False)
        self.register_buffer("sin_cached", emb.sin().to(dtype), persistent=False)

```

```

    def forward(self, x, seq_len=None):
        # x: [bs, num_attention_heads, seq_len, head_size]
        if self.max_seq_len_cached is None or seq_len > self.max_seq_len_cached:
            self._set_cos_sin_cache(seq_len=seq_len, device=x.device, dtype=x.dtype)

        return (
            self.cos_cached[:seq_len].to(dtype=x.dtype),
            self.sin_cached[:seq_len].to(dtype=x.dtype),
        )

```

```
# Copied from transformers.models.llama.modeling_llama.LlamaLinearScalingRotaryEmbedding with Llama->DeepseekV2
```

```
class DeepseekV2LinearScalingRotaryEmbedding(DeepseekV2RotaryEmbedding):
```

```
    """DeepseekV2RotaryEmbedding extended with linear scaling. Credits to the Reddit user /u/kaiokendev"""
```

```

    def __init__(
        self,
        dim,
        max_position_embeddings=2048,
        base=10000,
        device=None,
        scaling_factor=1.0,
    ):
        self.scaling_factor = scaling_factor
        super().__init__(dim, max_position_embeddings, base, device)

```

```

    def _set_cos_sin_cache(self, seq_len, device, dtype):
        self.max_seq_len_cached = seq_len
        t = torch.arange(

```

```

        self.max_seq_len_cached, device=device, dtype=self.inv_freq.dtype
    )
    t = t / self.scaling_factor

    freqs = torch.outer(t, self.inv_freq)
    # Different from paper, but it uses a different permutation in order to obtain the same calculation
    emb = torch.cat((freqs, freqs), dim=-1)
    self.register_buffer("cos_cached", emb.cos().to(dtype), persistent=False)
    self.register_buffer("sin_cached", emb.sin().to(dtype), persistent=False)

# Copied from transformers.models.llama.modeling_llama.LlamaDynamicNTKScalingRotaryEmbedding with Llama->DeepseekV2
class DeepseekV2DynamicNTKScalingRotaryEmbedding(DeepseekV2RotaryEmbedding):
    """DeepseekV2RotaryEmbedding extended with Dynamic NTK scaling. Credits to the Reddit users /u/bloc97 and
    /u/emozilla"""

    def __init__(
        self,
        dim,
        max_position_embeddings=2048,
        base=10000,
        device=None,
        scaling_factor=1.0,
    ):
        self.scaling_factor = scaling_factor
        super().__init__(dim, max_position_embeddings, base, device)

    def _set_cos_sin_cache(self, seq_len, device, dtype):
        self.max_seq_len_cached = seq_len

        if seq_len > self.max_position_embeddings:
            base = self.base * (
                (self.scaling_factor * seq_len / self.max_position_embeddings) - (self.scaling_factor - 1)
            ) ** (self.dim / (self.dim - 2))
            inv_freq = 1.0 / (
                base ** (torch.arange(0, self.dim, 2).float().to(device) / self.dim)
            )
            self.register_buffer("inv_freq", inv_freq, persistent=False)

        t = torch.arange(
            self.max_seq_len_cached, device=device, dtype=self.inv_freq.dtype
        )

        freqs = torch.outer(t, self.inv_freq)
        # Different from paper, but it uses a different permutation in order to obtain the same calculation
        emb = torch.cat((freqs, freqs), dim=-1)
        self.register_buffer("cos_cached", emb.cos().to(dtype), persistent=False)
        self.register_buffer("sin_cached", emb.sin().to(dtype), persistent=False)

# Inverse dim formula to find dim based on number of rotations
def yarn_find_correction_dim(
    num_rotations, dim, base=10000, max_position_embeddings=2048
):
    return (dim * math.log(max_position_embeddings / (num_rotations * 2 * math.pi))) / (
        2 * math.log(base)
    )

# Find dim range bounds based on rotations
def yarn_find_correction_range(
    low_rot, high_rot, dim, base=10000, max_position_embeddings=2048
):
    low = math.floor(
        yarn_find_correction_dim(low_rot, dim, base, max_position_embeddings)
    )
    high = math.ceil(
        yarn_find_correction_dim(high_rot, dim, base, max_position_embeddings)
    )
    return max(low, 0), min(high, dim - 1) # Clamp values just in case

```

```

def yarn_get_mscale(scale=1, mscale=1):
    if scale <= 1:
        return 1.0
    return 0.1 * mscale * math.log(scale) + 1.0

def yarn_linear_ramp_mask(min, max, dim):
    if min == max:
        max += 0.001 # Prevent singularity

    linear_func = (torch.arange(dim, dtype=torch.float32) - min) / (max - min)
    ramp_func = torch.clamp(linear_func, 0, 1)
    return ramp_func

class DeepseekV2YarnRotaryEmbedding(DeepseekV2RotaryEmbedding):

    def __init__(
        self,
        dim,
        max_position_embeddings=2048,
        base=10000,
        device=None,
        scaling_factor=1.0,
        original_max_position_embeddings=4096,
        beta_fast=32,
        beta_slow=1,
        mscale=1,
        mscale_all_dim=0,
    ):
        self.scaling_factor = scaling_factor
        self.original_max_position_embeddings = original_max_position_embeddings
        self.beta_fast = beta_fast
        self.beta_slow = beta_slow
        self.mscale = mscale
        self.mscale_all_dim = mscale_all_dim
        super().__init__(dim, max_position_embeddings, base, device)

    def _set_cos_sin_cache(self, seq_len, device, dtype):
        self.max_seq_len_cached = seq_len
        dim = self.dim

        freq_extra = 1.0 / (
            self.base
            ** (torch.arange(0, dim, 2, dtype=torch.float32, device=device) / dim)
        )
        freq_inter = 1.0 / (
            self.scaling_factor
            * self.base
            ** (torch.arange(0, dim, 2, dtype=torch.float32, device=device) / dim)
        )

        low, high = yarn_find_correction_range(
            self.beta_fast,
            self.beta_slow,
            dim,
            self.base,
            self.original_max_position_embeddings,
        )
        inv_freq_mask = 1.0 - yarn_linear_ramp_mask(low, high, dim // 2).to(
            device=device, dtype=torch.float32
        )
        inv_freq = freq_inter * (1 - inv_freq_mask) + freq_extra * inv_freq_mask
        self.register_buffer("inv_freq", inv_freq, persistent=False)

        t = torch.arange(seq_len, device=device, dtype=torch.float32)

        freqs = torch.outer(t, inv_freq)

```

```

        _mscale = float(
            yarn_get_mscale(self.scaling_factor, self.mscale)
            / yarn_get_mscale(self.scaling_factor, self.mscale_all_dim)
        )

        emb = torch.cat((freqs, freqs), dim=-1)
        self.register_buffer(
            "cos_cached", (emb.cos() * _mscale).to(dtype), persistent=False
        )
        self.register_buffer(
            "sin_cached", (emb.sin() * _mscale).to(dtype), persistent=False
        )

```

Copied from transformers.models.llama.modeling_llama.rotate_half

```

def rotate_half(x):
    """Rotates half the hidden dims of the input."""
    x1 = x[..., : x.shape[-1] // 2]
    x2 = x[..., x.shape[-1] // 2 :]
    return torch.cat((-x2, x1), dim=-1)

```

Copied from transformers.models.llama.modeling_llama.apply_rotary_pos_emb

```

def apply_rotary_pos_emb(q, k, cos, sin, position_ids, unsqueeze_dim=1):
    """Applies Rotary Position Embedding to the query and key tensors.

    Args:
        q (`torch.Tensor`): The query tensor.
        k (`torch.Tensor`): The key tensor.
        cos (`torch.Tensor`): The cosine part of the rotary embedding.
        sin (`torch.Tensor`): The sine part of the rotary embedding.
        position_ids (`torch.Tensor`):
            The position indices of the tokens corresponding to the query and key tensors. For example, this can be
            used to pass offsetted position ids when working with a KV-cache.
        unsqueeze_dim (`int`, *optional*, defaults to 1):
            The 'unsqueeze_dim' argument specifies the dimension along which to unsqueeze cos[position_ids] and
            sin[position_ids] so that they can be properly broadcasted to the dimensions of q and k. For example,
            note
            and
            have
            that cos[position_ids] and sin[position_ids] have the shape [batch_size, seq_len, head_dim]. Then, if q
            k have the shape [batch_size, heads, seq_len, head_dim], then setting unsqueeze_dim=1 makes
            cos[position_ids] and sin[position_ids] broadcastable to the shapes of q and k. Similarly, if q and k
            have
            the shape [batch_size, seq_len, heads, head_dim], then set unsqueeze_dim=2.

    Returns:
        `tuple(torch.Tensor)` comprising of the query and key tensors rotated using the Rotary Position Embedding.
    """
    cos = cos[position_ids].unsqueeze(unsqueeze_dim)
    sin = sin[position_ids].unsqueeze(unsqueeze_dim)

    # print()

    b, h, s, d = q.shape
    q = q.view(b, h, s, d // 2, 2).transpose(4, 3).reshape(b, h, s, d)

    b, h, s, d = k.shape
    k = k.view(b, h, s, d // 2, 2).transpose(4, 3).reshape(b, h, s, d)

    q_embed = (q * cos) + (rotate_half(q) * sin)
    k_embed = (k * cos) + (rotate_half(k) * sin)

    return q_embed, k_embed

```

class DeepseekV2MLP(nn.Module):

```

    def __init__(self, config, hidden_size=None, intermediate_size=None):
        super().__init__()
        self.config = config
        self.hidden_size = config.hidden_size if hidden_size is None else hidden_size

```

```

self.intermediate_size = (
    config.intermediate_size if intermediate_size is None else intermediate_size
)

```

```

self.gate_proj = nn.Linear(self.hidden_size, self.intermediate_size, bias=False)
self.up_proj = nn.Linear(self.hidden_size, self.intermediate_size, bias=False)
self.down_proj = nn.Linear(self.intermediate_size, self.hidden_size, bias=False)
self.act_fn = ACT2FN[config.hidden_act]

```

```

def forward(self, x):
    down_proj = self.down_proj(self.act_fn(self.gate_proj(x)) * self.up_proj(x))
    return down_proj

```

```

class MoEGate(nn.Module):

```

```

    def __init__(self, config):
        super().__init__()
        self.config = config
        self.top_k = config.num_experts_per_tok
        self.n_routed_experts = config.n_routed_experts
        self.routed_scaling_factor = config.routed_scaling_factor
        self.scoring_func = config.scoring_func
        self.alpha = config.aux_loss_alpha
        self.seq_aux = config.seq_aux
        self.topk_method = config.topk_method
        self.n_group = config.n_group
        self.topk_group = config.topk_group

```

```

        # topk selection algorithm
        self.norm_topk_prob = config.norm_topk_prob
        self.gating_dim = config.hidden_size
        self.weight = nn.Parameter(
            torch.empty((self.n_routed_experts, self.gating_dim))
        )
        if self.topk_method == "noaux_tc":
            self.e_score_correction_bias = nn.Parameter(
                torch.empty((self.n_routed_experts))
            )
        self.reset_parameters()

```

```

def reset_parameters(self) -> None:
    import torch.nn.init as init

    init.kaiming_uniform_(self.weight, a=math.sqrt(5))

```

```

def forward(self, hidden_states):
    bsz, seq_len, h = hidden_states.shape
    ### compute gating score
    hidden_states = hidden_states.view(-1, h)
    logits = F.linear(
        hidden_states.type(torch.float32), self.weight.type(torch.float32), None
    )
    if self.scoring_func == "softmax":
        scores = logits.softmax(dim=-1, dtype=torch.float32)
    elif self.scoring_func == "sigmoid":
        scores = logits.sigmoid()
    else:
        raise NotImplementedError(
            f"insupportable scoring function for MoE gating: {self.scoring_func}"
        )

    ### select top-k experts
    if self.topk_method == "greedy":
        topk_weight, topk_idx = torch.topk(
            scores, k=self.top_k, dim=-1, sorted=False
        )
    elif self.topk_method == "group_limited_greedy":
        group_scores = (
            scores.view(bsz * seq_len, self.n_group, -1).max(dim=-1).values
        ) # [n, n_group]
        group_idx = torch.topk(

```

```

        group_scores, k=self.topk_group, dim=-1, sorted=False
    )[
        1
    ] # [n, top_k_group]
    group_mask = torch.zeros_like(group_scores) # [n, n_group]
    group_mask.scatter_(1, group_idx, 1) # [n, n_group]
    score_mask = (
        group_mask.unsqueeze(-1)
        .expand(
            bsz * seq_len, self.n_group, self.n_routed_experts // self.n_group
        )
        .reshape(bsz * seq_len, -1)
    ) # [n, e]
    tmp_scores = scores.masked_fill(~score_mask.bool(), 0.0) # [n, e]
    topk_weight, topk_idx = torch.topk(
        tmp_scores, k=self.top_k, dim=-1, sorted=False
    )
elif self.topk_method == "noaux_tc":
    assert not self.training
    scores_for_choice = scores.view(bsz * seq_len, -1) + self.e_score_correction_bias.unsqueeze(0)
    group_scores = (
        scores_for_choice.view(bsz * seq_len, self.n_group, -1).topk(2, dim=-1)[0].sum(dim = -1)
    ) # [n, n_group]
    group_idx = torch.topk(
        group_scores, k=self.topk_group, dim=-1, sorted=False
    )[
        1
    ] # [n, top_k_group]
    group_mask = torch.zeros_like(group_scores) # [n, n_group]
    group_mask.scatter_(1, group_idx, 1) # [n, n_group]
    score_mask = (
        group_mask.unsqueeze(-1)
        .expand(
            bsz * seq_len, self.n_group, self.n_routed_experts // self.n_group
        )
        .reshape(bsz * seq_len, -1)
    ) # [n, e]
    tmp_scores = scores_for_choice.masked_fill(~score_mask.bool(), 0.0) # [n, e]
    _, topk_idx = torch.topk(
        tmp_scores, k=self.top_k, dim=-1, sorted=False
    )
    topk_weight = scores.gather(1, topk_idx)

### norm gate to sum 1
if self.top_k > 1 and self.norm_topk_prob:
    denominator = topk_weight.sum(dim=-1, keepdim=True) + 1e-20
    topk_weight = topk_weight / denominator * self.routed_scaling_factor
else:
    topk_weight = topk_weight * self.routed_scaling_factor
### expert-level computation auxiliary loss
if self.training and self.alpha > 0.0:
    scores_for_aux = scores
    aux_topk = self.top_k
    # always compute aux loss based on the naive greedy topk method
    topk_idx_for_aux_loss = topk_idx.view(bsz, -1)
    if self.seq_aux:
        scores_for_seq_aux = scores_for_aux.view(bsz, seq_len, -1)
        ce = torch.zeros(
            bsz, self.n_routed_experts, device=hidden_states.device
        )
        ce.scatter_add_(
            1,
            topk_idx_for_aux_loss,
            torch.ones(bsz, seq_len * aux_topk, device=hidden_states.device),
        ).div_(seq_len * aux_topk / self.n_routed_experts)
        aux_loss = (ce * scores_for_seq_aux.mean(dim=1)).sum(
            dim=1
        ).mean() * self.alpha
    else:
        mask_ce = F.one_hot(
            topk_idx_for_aux_loss.view(-1), num_classes=self.n_routed_experts

```

```

    )
    ce = mask_ce.float().mean(0)
    Pi = scores_for_aux.mean(0)
    fi = ce * self.n_routed_experts
    aux_loss = (Pi * fi).sum() * self.alpha
else:
    aux_loss = None
return topk_idx, topk_weight, aux_loss

```

```

class AddAuxiliaryLoss(torch.autograd.Function):
    """

```

```

    The trick function of adding auxiliary (aux) loss,
    which includes the gradient of the aux loss during backpropagation.
    """

```

```

    @staticmethod
    def forward(ctx, x, loss):
        assert loss.numel() == 1
        ctx.dtype = loss.dtype
        ctx.required_aux_loss = loss.requires_grad
        return x

    @staticmethod
    def backward(ctx, grad_output):
        grad_loss = None
        if ctx.required_aux_loss:
            grad_loss = torch.ones(1, dtype=ctx.dtype, device=grad_output.device)
        return grad_output, grad_loss

```

```

class DeepseekV2MoE(nn.Module):
    """

```

```

    A mixed expert module containing shared experts.
    """

```

```

    def __init__(self, config):
        super().__init__()
        self.config = config
        self.num_experts_per_tok = config.num_experts_per_tok

        if hasattr(config, "ep_size") and config.ep_size > 1:
            assert config.ep_size == dist.get_world_size()
            self.ep_size = config.ep_size
            self.experts_per_rank = config.n_routed_experts // config.ep_size
            self.ep_rank = dist.get_rank()
            self.experts = nn.ModuleList(
                [
                    (
                        DeepseekV2MLP(
                            config, intermediate_size=config.moe_intermediate_size
                        )
                        if i >= self.ep_rank * self.experts_per_rank
                        and i < (self.ep_rank + 1) * self.experts_per_rank
                        else None
                    )
                    for i in range(config.n_routed_experts)
                ]
            )
        else:
            self.ep_size = 1
            self.experts_per_rank = config.n_routed_experts
            self.ep_rank = 0
            self.experts = nn.ModuleList(
                [
                    DeepseekV2MLP(
                        config, intermediate_size=config.moe_intermediate_size
                    )
                    for i in range(config.n_routed_experts)
                ]
            )

```

```

self.gate = MoEGate(config)
if config.n_shared_experts is not None:
    intermediate_size = config.moe_intermediate_size * config.n_shared_experts
    self.shared_experts = DeepseekV2MLP(
        config=config, intermediate_size=intermediate_size
    )

def forward(self, hidden_states):
    identity = hidden_states
    orig_shape = hidden_states.shape
    topk_idx, topk_weight, aux_loss = self.gate(hidden_states)
    hidden_states = hidden_states.view(-1, hidden_states.shape[-1])
    flat_topk_idx = topk_idx.view(-1)
    if self.training:
        hidden_states = hidden_states.repeat_interleave(
            self.num_experts_per_tok, dim=0
        )
        y = torch.empty_like(hidden_states)
        for i, expert in enumerate(self.experts):
            y[flat_topk_idx == i] = expert(hidden_states[flat_topk_idx == i])
        y = (y.view(*topk_weight.shape, -1) * topk_weight.unsqueeze(-1)).sum(dim=1)
        y = y.to(hidden_states.dtype).view(*orig_shape)
        y = AddAuxiliaryLoss.apply(y, aux_loss)
    else:
        y = self.moe_infer(hidden_states, topk_idx, topk_weight).view(*orig_shape)
    if self.config.n_shared_experts is not None:
        y = y + self.shared_experts(identity)
    return y

@torch.no_grad()
def moe_infer(self, x, topk_ids, topk_weight):
    cnts = topk_ids.new_zeros((topk_ids.shape[0], len(self.experts)))
    cnts.scatter_(1, topk_ids, 1)
    tokens_per_expert = cnts.sum(dim=0)
    idxs = topk_ids.view(-1).argsort()
    sorted_tokens = x[idxs // topk_ids.shape[1]]
    sorted_tokens_shape = sorted_tokens.shape
    if self.ep_size > 1:
        tokens_per_ep_rank = tokens_per_expert.view(self.ep_size, -1).sum(dim=1)
        tokens_per_expert_group = tokens_per_expert.new_empty(
            tokens_per_expert.shape[0]
        )
        dist.all_to_all_single(tokens_per_expert_group, tokens_per_expert)
        output_splits = (
            tokens_per_expert_group.view(self.ep_size, -1)
            .sum(1)
            .cpu()
            .numpy()
            .tolist()
        )
        gathered_tokens = sorted_tokens.new_empty(
            tokens_per_expert_group.sum(dim=0).cpu().item(), sorted_tokens.shape[1]
        )
        input_split_sizes = tokens_per_ep_rank.cpu().numpy().tolist()
        dist.all_to_all(
            list(gathered_tokens.split(output_splits)),
            list(sorted_tokens.split(input_split_sizes)),
        )
        tokens_per_expert_post_gather = tokens_per_expert_group.view(
            self.ep_size, self.experts_per_rank
        ).sum(dim=0)
        gatherd_idxes = np.zeros(shape=(gathered_tokens.shape[0],), dtype=np.int32)
        s = 0
        for i, k in enumerate(tokens_per_expert_group.cpu().numpy()):
            gatherd_idxes[s : s + k] = i % self.experts_per_rank
            s += k
        gatherd_idxes = gatherd_idxes.argsort()
        sorted_tokens = gathered_tokens[gatherd_idxes]
        tokens_per_expert = tokens_per_expert_post_gather
    tokens_per_expert = tokens_per_expert.cpu().numpy()

```

```

outputs = []
start_idx = 0
for i, num_tokens in enumerate(tokens_per_expert):
    end_idx = start_idx + num_tokens
    if num_tokens == 0:
        continue
    expert = self.experts[i + self.ep_rank * self.experts_per_rank]
    tokens_for_this_expert = sorted_tokens[start_idx:end_idx]
    expert_out = expert(tokens_for_this_expert)
    outputs.append(expert_out)
    start_idx = end_idx

outs = torch.cat(outputs, dim=0) if len(outputs) else sorted_tokens.new_empty(0)
if self.ep_size > 1:
    new_x = torch.empty_like(outs)
    new_x[gatherd_idxs] = outs
    gathered_tokens = new_x.new_empty(*sorted_tokens.shape)
    dist.all_to_all(
        list(gathered_tokens.split(input_split_sizes)),
        list(new_x.split(output_splits)),
    )
    outs = gathered_tokens

new_x = torch.empty_like(outs)
new_x[idxs] = outs
final_out = (
    new_x.view(*topk_ids.shape, -1)
    .type(topk_weight.dtype)
    .mul_(topk_weight.unsqueeze(dim=-1))
    .sum(dim=1)
    .type(new_x.dtype)
)
return final_out

```

Copied from transformers.models.llama.modeling_llama.repeat_kv

def repeat_kv(hidden_states: torch.Tensor, n_rep: int) -> torch.Tensor:

"""
 This is the equivalent of torch.repeat_interleave(x, dim=1, repeats=n_rep). The hidden states go from (batch,
 num_key_value_heads, seqlen, head_dim) to (batch, num_attention_heads, seqlen, head_dim)
 """

batch, num_key_value_heads, slen, head_dim = hidden_states.shape

if n_rep == 1:

return hidden_states

hidden_states = hidden_states[:, :, None, :, :].expand(
 batch, num_key_value_heads, n_rep, slen, head_dim
)

return hidden_states.reshape(batch, num_key_value_heads * n_rep, slen, head_dim)

Copied from transformers.models.llama.modeling_llama.LlamaAttention with Llama->DeepseekV2

class DeepseekV2Attention(nn.Module):

"""Multi-headed attention from 'Attention Is All You Need' paper"""

def __init__(self, config: DeepseekV2Config, layer_idx: Optional[int] = None):

super().__init__()

self.config = config

self.layer_idx = layer_idx

if layer_idx is None:

logger.warning_once(

f"Instantiating {self.__class__.__name__} without passing `layer\_idx` is not recommended and will "

"to errors during the forward call, if caching is used. Please make sure to provide a `layer\_idx` "

"when creating this class."

)

self.attention_dropout = config.attention_dropout

self.hidden_size = config.hidden_size

self.num_heads = config.num_attention_heads

self.max_position_embeddings = config.max_position_embeddings

self.rope_theta = config.rope_theta

```

self.q_lora_rank = config.q_lora_rank
self.qk_rope_head_dim = config.qk_rope_head_dim
self.kv_lora_rank = config.kv_lora_rank
self.v_head_dim = config.v_head_dim
self.qk_nope_head_dim = config.qk_nope_head_dim
self.q_head_dim = config.qk_nope_head_dim + config.qk_rope_head_dim

self.is_causal = True

if self.q_lora_rank is None:
    self.q_proj = nn.Linear(
        self.hidden_size, self.num_heads * self.q_head_dim, bias=False
    )
else:
    self.q_a_proj = nn.Linear(
        self.hidden_size, config.q_lora_rank, bias=config.attention_bias
    )
    self.q_a_layernorm = DeepseekV2RMSNorm(config.q_lora_rank)
    self.q_b_proj = nn.Linear(
        config.q_lora_rank, self.num_heads * self.q_head_dim, bias=False
    )
# config.kv_lora_rank + config.qk_rope_head_dim,
self.kv_a_proj_with_mqa = nn.Linear(
    self.hidden_size,
    config.kv_lora_rank + config.qk_rope_head_dim,
    bias=config.attention_bias,
)
self.kv_a_layernorm = DeepseekV2RMSNorm(config.kv_lora_rank)
self.kv_b_proj = nn.Linear(
    config.kv_lora_rank,
    self.num_heads
    * (self.q_head_dim - self.qk_rope_head_dim + self.v_head_dim),
    bias=False,
)

self.o_proj = nn.Linear(
    self.num_heads * self.v_head_dim,
    self.hidden_size,
    bias=config.attention_bias,
)
self._init_rope()

self.softmax_scale = self.q_head_dim ** (-0.5)
if self.config.rope_scaling is not None:
    mscale_all_dim = self.config.rope_scaling.get("mscale_all_dim", 0)
    scaling_factor = self.config.rope_scaling["factor"]
    if mscale_all_dim:
        mscale = yarn_get_mscale(scaling_factor, mscale_all_dim)
        self.softmax_scale = self.softmax_scale * mscale * mscale

def _init_rope(self):
    if self.config.rope_scaling is None:
        self.rotary_emb = DeepseekV2RotaryEmbedding(
            self.qk_rope_head_dim,
            max_position_embeddings=self.max_position_embeddings,
            base=self.rope_theta,
        )
        # self.rotary_emb = DeepseekV2LinearScalingRotaryEmbedding(
        #     self.qk_rope_head_dim,
        #     max_position_embeddings=self.max_position_embeddings,
        #     scaling_factor=scaling_factor,
        #     base=self.rope_theta,
        # )
    else:
        scaling_type = self.config.rope_scaling["type"]
        scaling_factor = self.config.rope_scaling["factor"]
        if scaling_type == "linear":
            self.rotary_emb = DeepseekV2LinearScalingRotaryEmbedding(
                self.qk_rope_head_dim,
                max_position_embeddings=self.max_position_embeddings,
                scaling_factor=scaling_factor,

```

```

        base=self.rope_theta,
    )
    elif scaling_type == "dynamic":
        self.rotary_emb = DeepseekV2DynamicNTKScalingRotaryEmbedding(
            self.qk_rope_head_dim,
            max_position_embeddings=self.max_position_embeddings,
            scaling_factor=scaling_factor,
            base=self.rope_theta,
        )
    elif scaling_type == "yarn":
        kwargs = {
            key: self.config.rope_scaling[key]
            for key in [
                "original_max_position_embeddings",
                "beta_fast",
                "beta_slow",
                "mscale",
                "mscale_all_dim",
            ]
            if key in self.config.rope_scaling
        }
        self.rotary_emb = DeepseekV2YarnRotaryEmbedding(
            self.qk_rope_head_dim,
            max_position_embeddings=self.max_position_embeddings,
            scaling_factor=scaling_factor,
            base=self.rope_theta,
            **kwargs,
        )
    else:
        raise ValueError(f"Unknown RoPE scaling type {scaling_type}")

def _shape(self, tensor: torch.Tensor, seq_len: int, bsz: int):
    return (
        tensor.view(bsz, seq_len, self.num_heads, self.v_head_dim)
        .transpose(1, 2)
        .contiguous()
    )

def forward(
    self,
    hidden_states: torch.Tensor,
    attention_mask: Optional[torch.Tensor] = None,
    position_ids: Optional[torch.LongTensor] = None,
    past_key_value: Optional[Cache] = None,
    output_attentions: bool = False,
    use_cache: bool = False,
    **kwargs,
) -> Tuple[torch.Tensor, Optional[torch.Tensor], Optional[Tuple[torch.Tensor]]]:
    if "padding_mask" in kwargs:
        warnings.warn(
            "Passing `padding_mask` is deprecated and will be removed in v4.37. Please make sure use `attention_mask` instead."
        )
    bsz, q_len, _ = hidden_states.size()

    if self.q_lora_rank is None:
        q = self.q_proj(hidden_states)
    else:
        q = self.q_b_proj(self.q_a_layernorm(self.q_a_proj(hidden_states)))
    q = q.view(bsz, q_len, self.num_heads, self.q_head_dim).transpose(1, 2)

    q_nope, q_pe = torch.split(
        q, [self.qk_nope_head_dim, self.qk_rope_head_dim], dim=-1
    )

    compressed_kv = self.kv_a_proj_with_mqa(hidden_states)
    compressed_kv, k_pe = torch.split(
        compressed_kv, [self.kv_lora_rank, self.qk_rope_head_dim], dim=-1
    )
    compressed_kv = self.kv_a_layernorm(compressed_kv)

```

```

k_pe = k_pe.view(bsz, q_len, 1, self.qk_rope_head_dim).transpose(1, 2)

kv_seq_len = k_pe.shape[-2]
if past_key_value is not None:
    if self.layer_idx is None:
        raise ValueError(
            f"The cache structure has changed since version v4.36. If you are using
{self.__class__.__name__} "
            "for auto-regressive decoding with k/v caching, please make sure to initialize the attention
class "
            "with a layer index."
        )
    kv_seq_len += past_key_value.get_usable_length(kv_seq_len, self.layer_idx)

cos, sin = self.rotary_emb(q_pe, seq_len=kv_seq_len)
q_pe, k_pe = apply_rotary_pos_emb(q_pe, k_pe, cos, sin, position_ids)

if past_key_value is not None:
    cache_kwargs = {"sin": sin, "cos": cos} # Specific to RoPE models
    compressed_kv = compressed_kv.unsqueeze(1)
    k_pe, compressed_kv = past_key_value.update(k_pe, compressed_kv, self.layer_idx, cache_kwargs)
    compressed_kv = compressed_kv.squeeze(1)

kv_b_proj = self.kv_b_proj.weight.view(self.num_heads, -1, self.kv_lora_rank)
q_absorb = kv_b_proj[:, :self.qk_nope_head_dim, :]
out_absorb = kv_b_proj[:, self.qk_nope_head_dim:, :]

q_nope = torch.matmul(q_nope, q_absorb)
attn_weights = (torch.matmul(q_pe, k_pe.mT) +
                 torch.matmul(q_nope, compressed_kv.unsqueeze(-3).mT)) * self.softmax_scale
if attn_weights.size() != (bsz, self.num_heads, q_len, kv_seq_len):
    raise ValueError(
        f"Attention weights should be of size {(bsz, self.num_heads, q_len, kv_seq_len)}, but is"
        f" {attn_weights.size()}"
    )
assert attention_mask is not None
if attention_mask is not None:
    if attention_mask.size() != (bsz, 1, q_len, kv_seq_len):
        raise ValueError(
            f"Attention mask should be of size {(bsz, 1, q_len, kv_seq_len)}, but is"
            f" {attention_mask.size()}"
        )
    attn_weights = attn_weights + attention_mask

# upcast attention to fp32
attn_weights = nn.functional.softmax(
    attn_weights, dim=-1, dtype=torch.float32
).to(q_pe.dtype)
attn_weights = nn.functional.dropout(
    attn_weights, p=self.attention_dropout, training=self.training
)
attn_output = torch.einsum('bhql,bkc->bhqc', attn_weights, compressed_kv)

attn_output = torch.matmul(attn_output, out_absorb.mT)

if attn_output.size() != (bsz, self.num_heads, q_len, self.v_head_dim):
    raise ValueError(
        f"`attn_output` should be of size {(bsz, self.num_heads, q_len, self.v_head_dim)}, but is"
        f" {attn_output.size()}"
    )

attn_output = attn_output.transpose(1, 2).contiguous()

attn_output = attn_output.reshape(bsz, q_len, self.num_heads * self.v_head_dim)

attn_output = self.o_proj(attn_output)

if not output_attentions:
    attn_weights = None

return attn_output, attn_weights, past_key_value

```

```

# Copied from transformers.models.llama.modeling_llama.LlamaFlashAttention2 with Llama->DeepseekV2
class DeepseekV2FlashAttention2(DeepseekV2Attention):
    """
    DeepseekV2 flash attention module. This module inherits from `DeepseekV2Attention` as the weights of the module
    stays
    untouched. The only required change would be on the forward pass where it needs to correctly call the public
    API of
    flash attention and deal with padding tokens in case the input contains any of them.
    """

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

        # TODO: Should be removed once Flash Attention for RoCm is bumped to 2.1.
        # flash_attn<2.1 generates top-left aligned causal mask, while what is needed here is bottom-right
        alignment, that was made default for flash_attn>=2.1. This attribute is used to handle this difference. Reference:
        ***
        # Beware that with flash_attn<2.1, using q_seqlen != k_seqlen (except for the case q_seqlen == 1) produces
        a wrong mask (top-left).
        self._flash_attn_uses_top_left_mask = not is_flash_attn_greater_or_equal_2_10()

    def forward(
        self,
        hidden_states: torch.Tensor,
        attention_mask: Optional[torch.LongTensor] = None,
        position_ids: Optional[torch.LongTensor] = None,
        past_key_value: Optional[Cache] = None,
        output_attentions: bool = False,
        use_cache: bool = False,
        **kwargs,
    ) -> Tuple[torch.Tensor, Optional[torch.Tensor], Optional[Tuple[torch.Tensor]]]:
        # DeepseekV2FlashAttention2 attention does not support output_attentions
        if "padding_mask" in kwargs:
            warnings.warn(
                "Passing `padding_mask` is deprecated and will be removed in v4.37. Please make sure use "
                "`attention_mask` instead."
            )

        # overwrite attention_mask with padding_mask
        attention_mask = kwargs.pop("padding_mask")

        output_attentions = False

        bsz, q_len, _ = hidden_states.size()

        if self.q_lora_rank is None:
            q = self.q_proj(hidden_states)
        else:
            q = self.q_b_proj(self.q_a_layernorm(self.q_a_proj(hidden_states)))
            q = q.view(bsz, q_len, self.num_heads, self.q_head_dim).transpose(1, 2)
            q_nope, q_pe = torch.split(
                q, [self.qk_nope_head_dim, self.qk_rope_head_dim], dim=-1
            )

        # Flash attention requires the input to have the shape
        # batch_size x seq_length x head_dim x hidden_dim
        # therefore we just need to keep the original shape
        compressed_kv = self.kv_a_proj_with_mqa(hidden_states)
        compressed_kv, k_pe = torch.split(
            compressed_kv, [self.kv_lora_rank, self.qk_rope_head_dim], dim=-1
        )
        k_pe = k_pe.view(bsz, q_len, 1, self.qk_rope_head_dim).transpose(1, 2)
        kv = (
            self.kv_b_proj(self.kv_a_layernorm(compressed_kv))
            .view(bsz, q_len, self.num_heads, self.qk_nope_head_dim + self.v_head_dim)
            .transpose(1, 2)
        )

        k_nope, value_states = torch.split(

```

```

        kv, [self.qk_nope_head_dim, self.v_head_dim], dim=-1
    )
    kv_seq_len = value_states.shape[-2]

    kv_seq_len = value_states.shape[-2]
    if past_key_value is not None:
        kv_seq_len += past_key_value.get_usable_length(kv_seq_len, self.layer_idx)

    cos, sin = self.rotary_emb(value_states, seq_len=kv_seq_len)
    q_pe, k_pe = apply_rotary_pos_emb(q_pe, k_pe, cos, sin, position_ids)

    query_states = k_pe.new_empty(bsz, self.num_heads, q_len, self.q_head_dim)
    query_states[:, :, :, : self.qk_nope_head_dim] = q_nope
    query_states[:, :, :, self.qk_nope_head_dim :] = q_pe

    key_states = k_pe.new_empty(bsz, self.num_heads, q_len, self.q_head_dim)
    key_states[:, :, :, : self.qk_nope_head_dim] = k_nope
    key_states[:, :, :, self.qk_nope_head_dim :] = k_pe

    if self.q_head_dim != self.v_head_dim:
        value_states = F.pad(value_states, [0, self.q_head_dim - self.v_head_dim])

    # TODO: support compressed_kv for kv_cache (instead of key_states, value_states) in flash_attention version
    if past_key_value is not None:
        cache_kwargs = {"sin": sin, "cos": cos} # Specific to RoPE models
        key_states, value_states = past_key_value.update(
            key_states, value_states, self.layer_idx, cache_kwargs
        )

    # TODO: These transpose are quite inefficient but Flash Attention requires the layout [batch_size,
    sequence_length, num_heads, head_dim]. We would need to refactor the KV cache
    # to be able to avoid many of these transpose/reshape/view.
    query_states = query_states.transpose(1, 2)
    key_states = key_states.transpose(1, 2)
    value_states = value_states.transpose(1, 2)

    dropout_rate = self.attention_dropout if self.training else 0.0

    # In PEFT, usually we cast the layer norms in float32 for training stability reasons
    # therefore the input hidden states gets silently casted in float32. Hence, we need
    # cast them back in the correct dtype just to be sure everything works as expected.
    # This might slowdown training & inference so it is recommended to not cast the LayerNorms
    # in fp32. (DeepseekV2RMSNorm handles it correctly)

    input_dtype = query_states.dtype
    if input_dtype == torch.float32:
        # Handle the case where the model is quantized
        if hasattr(self.config, "_pre_quantization_dtype"):
            target_dtype = self.config._pre_quantization_dtype
        elif torch.is_autocast_enabled():
            target_dtype = torch.get_autocast_gpu_dtype()
        else:
            target_dtype = (
                self.q_proj.weight.dtype
                if self.q_lora_rank is None
                else self.q_a_proj.weight.dtype
            )

        logger.warning_once(
            f"The input hidden states seems to be silently casted in float32, this might be related to"
            f" the fact you have upcasted embedding or layer norm layers in float32. We will cast back the"
            f" {target_dtype}."
        )

    query_states = query_states.to(target_dtype)
    key_states = key_states.to(target_dtype)
    value_states = value_states.to(target_dtype)

    attn_output = self._flash_attention_forward(
        query_states,

```

```

        key_states,
        value_states,
        attention_mask,
        q_len,
        dropout=dropout_rate,
        softmax_scale=self.softmax_scale,
    )
    if self.q_head_dim != self.v_head_dim:
        attn_output = attn_output[:, :, :, : self.v_head_dim]

    attn_output = attn_output.reshape(
        bsz, q_len, self.num_heads * self.v_head_dim
    ).contiguous()
    attn_output = self.o_proj(attn_output)

    if not output_attentions:
        attn_weights = None

    return attn_output, attn_weights, past_key_value

```

```

def _flash_attention_forward(
    self,
    query_states,
    key_states,
    value_states,
    attention_mask,
    query_length,
    dropout=0.0,
    softmax_scale=None,
):
    """

```

Calls the forward method of Flash Attention - if the input hidden states contain at least one padding token first unpad the input, then computes the attention scores and pad the final attention scores.

Args:

```

    query_states (`torch.Tensor`):
        Input query states to be passed to Flash Attention API
    key_states (`torch.Tensor`):
        Input key states to be passed to Flash Attention API
    value_states (`torch.Tensor`):
        Input value states to be passed to Flash Attention API
    attention_mask (`torch.Tensor`):
        The padding mask - corresponds to a tensor of size `(batch_size, seq_len)` where 0 stands for the
        position of padding tokens and 1 for the position of non-padding tokens.
    dropout (`int`, *optional*):
        Attention dropout
    softmax_scale (`float`, *optional*):
        The scaling of QK^T before applying softmax. Default to 1 / sqrt(head_dim)
    """

```

```

    if not self._flash_attn_uses_top_left_mask:
        causal = self.is_causal
    else:

```

TODO: Remove the `query\_length != 1` check once Flash Attention for RoCm is bumped to 2.1. For details, please see the comment in DeepseekV2FlashAttention2 __init__.

```

        causal = self.is_causal and query_length != 1

```

Contains at least one padding token in the sequence

```

    if attention_mask is not None:
        batch_size = query_states.shape[0]
        (
            query_states,
            key_states,
            value_states,
            indices_q,
            cu_seq_lens,
            max_seq_lens,
        ) = self._upad_input(
            query_states, key_states, value_states, attention_mask, query_length
        )

```

```

    cu_seq_lens_q, cu_seq_lens_k = cu_seq_lens

```

```

max_seqlen_in_batch_q, max_seqlen_in_batch_k = max_seq_lens

attn_output_unpad = flash_attn_varlen_func(
    query_states,
    key_states,
    value_states,
    cu_seqlens_q=cu_seqlens_q,
    cu_seqlens_k=cu_seqlens_k,
    max_seqlen_q=max_seqlen_in_batch_q,
    max_seqlen_k=max_seqlen_in_batch_k,
    dropout_p=dropout,
    softmax_scale=softmax_scale,
    causal=causal,
)

attn_output = pad_input(
    attn_output_unpad, indices_q, batch_size, query_length
)
else:
    attn_output = flash_attn_func(
        query_states,
        key_states,
        value_states,
        dropout,
        softmax_scale=softmax_scale,
        causal=causal,
    )

return attn_output

def _upad_input(
    self, query_layer, key_layer, value_layer, attention_mask, query_length
):
    indices_k, cu_seqlens_k, max_seqlen_in_batch_k = _get_unpad_data(attention_mask)
    batch_size, kv_seq_len, num_key_value_heads, head_dim = key_layer.shape

    key_layer = index_first_axis(
        key_layer.reshape(batch_size * kv_seq_len, num_key_value_heads, head_dim),
        indices_k,
    )
    value_layer = index_first_axis(
        value_layer.reshape(batch_size * kv_seq_len, num_key_value_heads, head_dim),
        indices_k,
    )
    if query_length == kv_seq_len:
        query_layer = index_first_axis(
            query_layer.reshape(batch_size * kv_seq_len, self.num_heads, head_dim),
            indices_k,
        )
        cu_seqlens_q = cu_seqlens_k
        max_seqlen_in_batch_q = max_seqlen_in_batch_k
        indices_q = indices_k
    elif query_length == 1:
        max_seqlen_in_batch_q = 1
        cu_seqlens_q = torch.arange(
            batch_size + 1, dtype=torch.int32, device=query_layer.device
        ) # There is a memcpy here, that is very bad.
        indices_q = cu_seqlens_q[:-1]
        query_layer = query_layer.squeeze(1)
    else:
        # The -q_len: slice assumes left padding.
        attention_mask = attention_mask[:, -query_length:]
        query_layer, indices_q, cu_seqlens_q, max_seqlen_in_batch_q = _unpad_input(
            query_layer, attention_mask
        )

    return (
        query_layer,
        key_layer,
        value_layer,
        indices_q,

```

```

        (cu_seqlens_q, cu_seqlens_k),
        (max_seqlen_in_batch_q, max_seqlen_in_batch_k),
    )

ATTENTION_CLASSES = {
    "eager": DeepseekV2Attention,
    "flash_attention_2": DeepseekV2FlashAttention2,

    "mla_eager": DeepseekV2Attention,
    "mla_flash_attention_2": DeepseekV2FlashAttention2,

    "mha_eager": LlamaAttention,
    "mha_flash_attention_2": LlamaFlashAttention2
}

class DeepseekV2DecoderLayer(nn.Module):
    def __init__(self, config: DeepseekV2Config, layer_idx: int):
        super().__init__()
        self.hidden_size = config.hidden_size

        if config.use_mla:
            attn_implementation = "mla_" + config._attn_implementation
        else:
            attn_implementation = "mha_" + config._attn_implementation

        self.self_attn = ATTENTION_CLASSES[attn_implementation](
            config=config, layer_idx=layer_idx
        )

        self.mlp = (
            DeepseekV2MoE(config)
            if (
                config.n_routed_experts is not None
                and layer_idx >= config.first_k_dense_replace
                and layer_idx % config.moe_layer_freq == 0
            )
            else DeepseekV2MLP(config)
        )
        self.input_layernorm = DeepseekV2RMSNorm(
            config.hidden_size, eps=config.rms_norm_eps
        )
        self.post_attention_layernorm = DeepseekV2RMSNorm(
            config.hidden_size, eps=config.rms_norm_eps
        )

    def forward(
        self,
        hidden_states: torch.Tensor,
        attention_mask: Optional[torch.Tensor] = None,
        position_ids: Optional[torch.LongTensor] = None,
        past_key_value: Optional[Tuple[torch.Tensor]] = None,
        output_attentions: Optional[bool] = False,
        use_cache: Optional[bool] = False,
        **kwargs,
    ) -> Tuple[
        torch.FloatTensor, Optional[Tuple[torch.FloatTensor, torch.FloatTensor]]
    ]:
        """
        Args:
            hidden_states (`torch.FloatTensor`): in

```

models/DeepSeek-OCR-2/processor_config.json

```

{
  "add_special_token": false,
  "candidate_resolutions": [

```

```

[
  1024,
  1024
],
"downsample_ratio": 4,
"ignore_id": -100,
"image_mean": [
  0.5,
  0.5,
  0.5
],
"image_std": [
  0.5,
  0.5,
  0.5
],
"image_token": "<image>",
"mask_prompt": false,
"normalize": true,
"pad_token": "<\uff5c\u2581pad\u2581\uff5c>",
"patch_size": 16,
"processor_class": "DeepseekVLV2Processor",
"sft_format": "deepseek"
}

```

sql/schema.sql

```

CREATE TYPE task_status AS ENUM (
  'queued',
  'running',
  'succeeded',
  'failed',
  'canceled'
);

CREATE TABLE IF NOT EXISTS ocr_files (
  id VARCHAR(64) PRIMARY KEY,
  original_name VARCHAR(255) NOT NULL,
  storage_path VARCHAR(1024) NOT NULL,
  content_type VARCHAR(255),
  size_bytes INTEGER NOT NULL,
  sha256 VARCHAR(64) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS ix_ocr_files_sha256
  ON ocr_files (sha256);

CREATE TABLE IF NOT EXISTS ocr_tasks (
  id VARCHAR(64) PRIMARY KEY,
  file_id VARCHAR(64) NOT NULL REFERENCES ocr_files(id),
  mode VARCHAR(64) NOT NULL DEFAULT 'document_markdown',
  status task_status NOT NULL DEFAULT 'queued',
  options JSONB NOT NULL DEFAULT '{}'::jsonb,
  priority INTEGER NOT NULL DEFAULT 100,
  attempts INTEGER NOT NULL DEFAULT 0,
  max_attempts INTEGER NOT NULL DEFAULT 3,
  worker_id VARCHAR(128),
  locked_at TIMESTAMPTZ,
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  error_message TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS ix_ocr_tasks_file_id
  ON ocr_tasks (file_id);

```

```
CREATE INDEX IF NOT EXISTS ix_ocr_tasks_status
  ON ocr_tasks (status);

CREATE INDEX IF NOT EXISTS ix_ocr_tasks_created_at
  ON ocr_tasks (created_at);

CREATE INDEX IF NOT EXISTS ix_ocr_tasks_queue_claim
  ON ocr_tasks (status, priority, created_at);

CREATE TABLE IF NOT EXISTS ocr_results (
  id VARCHAR(64) PRIMARY KEY,
  task_id VARCHAR(64) NOT NULL UNIQUE REFERENCES ocr_tasks(id),
  text TEXT NOT NULL DEFAULT '',
  markdown TEXT NOT NULL DEFAULT '',
  layout JSONB NOT NULL DEFAULT '{} '::jsonb,
  metadata_json JSONB NOT NULL DEFAULT '{} '::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```